

先進計算機構成論 06

東京大学大学院 情報理工学系研究科 創造情報学専攻

塩谷 亮太

shioya@ci.i.u-tokyo.ac.jp

前回のおさらい：分岐予測器の基本（１）

- 分岐予測では、以下の３つを全て行う必要がある
 1. 分岐命令かどうか予測（分岐種別の予測）
 2. 分岐先ターゲット予測
 3. 分岐方向予測
- if-then-else の方向だけを予測していれば良いわけではない

前回のおさらい：分岐予測器の基本（2）

1. 命令パイプラインと性能
2. 分岐予測（前編）
 1. 分岐命令かどうか予測（分岐種別の予測）
 2. 分岐先ターゲット予測
 3. 分岐方向予測
 1. 静的予測
 2. 動的予測

今日の内容

1. 予測精度と性能の簡単な見積もり
2. 間接分岐予測器
3. 高度な方向予測器
4. フェッチパイプライン

予測精度と性能の簡単な見積もり

分岐予測ミスによる実行サイクルの増加のモデル

■ 以下のようにおいた場合,

- ミスがない理想的な実行の際のサイクル数 : C_t
- 分岐予測ミスの発生回数 : $N_m = N_i \times P_b \times P_m$
 - 実行命令数 : N_i
 - プログラム中の分岐命令の出現率 : P_b
 - 分岐命令毎の予測ミス発生率 : P_m
- 分岐予測ミス・ペナルティ : C_p

■ 実行サイクル数 C_r は, 理想サイクル数 C_t に対して,

- $C_r = C_t + N_m \times C_p$
- 意味 : 予測ミスの発生回数×ペナルティ だけ実行時間が伸びる

具体的な値を入れてみる

- 以下のように置いた場合,
 - 理想的な実行の際のサイクル数 : $Ct = 1M$ (メガ)
 - 分岐予測ミスの発生回数 : $Nm = Ni \times Pb \times Pm = 0.02M$
 - 実行命令数 : $Ni = 8M$
 - ◇ 最近の CPU は 1 サイクルあたり 8 命令ぐらいは本当に実行できる
 - 分岐命令の出現率 : $Pb = 0.25$
 - ◇ プログラム中には大体 4 命令に 1 つぐらい分岐が出てくる
 - 分岐命令毎の予測ミス発生率 : $Pm = 0.01$
 - ◇ 仮に 99% 精度で予測できると仮定
 - 分岐予測ミス・ペナルティ : $Cp = 15$
- 実行サイクル数 Cr は
 - $Cr = Ct + Nm \times Cp = 1M + 0.3M = 1.3M$
 - 99% 精度で予測をあててもなお, 30%速度を低下させている

この結果からわかること

- ベースラインの CPU の性能が上がるほど、予測精度も重要になる
 - わずかな予測精度向上が性能に寄与する
- ミスパナルティの大きさは性能に大きく影響する

間接分岐予測

間接分岐命令

■ 方向分岐命令

- 成立 or 不成立で行き先が分岐
- `bne x1, x2, TARGET`
- `if` 文に相当

■ 間接分岐命令

- レジスタに格納されているアドレスに飛ぶ
- 「`jr x1`」は, `x1` レジスタに格納されている値のアドレスにジャンプする間接分岐命令

間接分岐命令の使いどころ

- 関数ポインタ呼び出し, 仮想関数呼び出し
 - 関数ポインタ変数やオブジェクト内の仮想関数ポインタをロード
 - ロードしたアドレスに間接分岐で飛ぶ
- switch-case 文
 - 例 : 0 ~ 255 の 256 のパターンの case がある場合
 - 各 case の処理の先頭の命令アドレスを要素数 256 の配列に格納
 - 配列の要素をロードして, そこに間接分岐で飛ぶ
- return 文
 - 関数呼び出し時に, 呼び出し元をメモリ上に保存
 - 呼び出し元アドレスをメモリからロードして, 間接分岐で戻る

間接分岐予測

1. 間接分岐一般の予測
2. 関数からの return の予測

間接分岐一般の予測

- 基本的に間接分岐の予測は難しい
- 方向分岐：
 - 成立 or 不成立の2択を予測
 - 分岐先ターゲット・アドレスは不変
- 間接分岐：
 - 動的に変化する, 無数にある分岐先アドレスを予測
 - 間接分岐の飛び先は, ロードした結果など = 64 bit などの値

簡単なもの：BTB を使った間接分岐の予測

- 基本的には、「前回飛んだ先に今回も飛ぶ」と予測
- BTB に「分岐の種類」を表す 1 ビットを格納して予測する
 - 0：方向分岐 →
 - これまで説明した方法で予測
 - 1：間接分岐 →
 - 方向分岐予測の結果は無視
 - BTB の読み出し結果をそのまま使う
- BTB には前回に間接分岐を実行した際の飛び先が入っている
 - これを読み出してそのまま飛び先として予測

もう少し凝ったもの

- BTB を引くときのアドレスに履歴をまぜる
 - 方向分岐の, グローバル分岐履歴を使うものと同じ
 - 過去に辿ってきた分岐方向によって飛び先を変えることができる
 - ARM Cortex A15 など搭載
- より発展させたものとしては, TAGE 予測器を応用したものなどが提案されている
 - TAGE のカウンタの部分にターゲットアドレスが入っている感じ

間接分岐予測器の効果

- Python のようなインタプリタの性能を大きく改善する
- *E. Rohou et al., Branch prediction and the performance of interpreters — Don't trust folklore, CGO 2015*
 - Intel プロセッサ（Nehalem, Sandy Bridge, Haswell）と ITTAGE 予測器等の比較
 - 世代が進むともものすごく精度が改善している

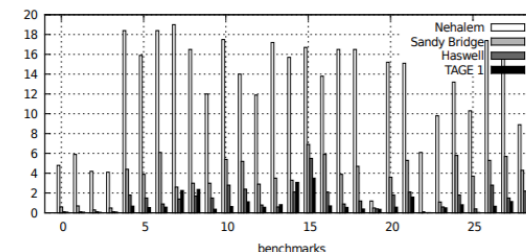


Figure 4. Python MPKI for all predictors

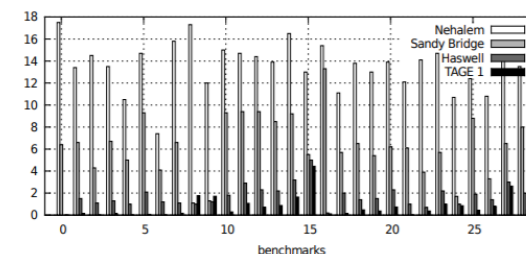


Figure 5. Javascript MPKI for all predictors

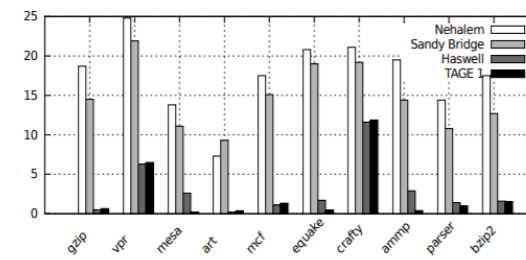


Figure 6. CLI MPKI for all predictors

間接分岐予測

1. 間接分岐一般の予測
2. 関数からの `return` の予測

関数からの return の予測

- 関数の call
 - 関数の先頭アドレスは固定
 - したがって、飛び先も固定
- 関数からの return
 - 同じ関数でも、呼び出し元は無数に存在
 - 直前に実行された call に戻ると期待できる
- 上記の性質を使って、return は高精度に予測できる

Return Address Stack (RAS) を使った予測

■ 使用する構造 :

- RAS :
 - 関数の呼び出し元を格納するスタック
- BTB :
 - BTB の「分岐の種類」を表すビットを拡張
 - ◇ 関数コール or リターンであることを記録
 - ◇ デコードするまで, 関数呼び出しであることはわからない

■ 動作 :

- BTB 引きの結果が関数コール :
 - スタックにその時の PC + 4 (戻り先) を積む
- BTB 引きの結果がリターン :
 - スタックのトップを読んで, そこに戻ると予測

高度な方向予測器

高度な分岐方向予測

1. 履歴を用いた方向予測器
 1. ローカル履歴予測器
 2. グローバル履歴予測器
 1. g-share 予測器
2. より高度な履歴を用いた予測器
 1. パーセプトロン 予測器
 2. TAGE 予測器
 3. RUNLTS 予測器

N ビット・カウンタ予測器の問題

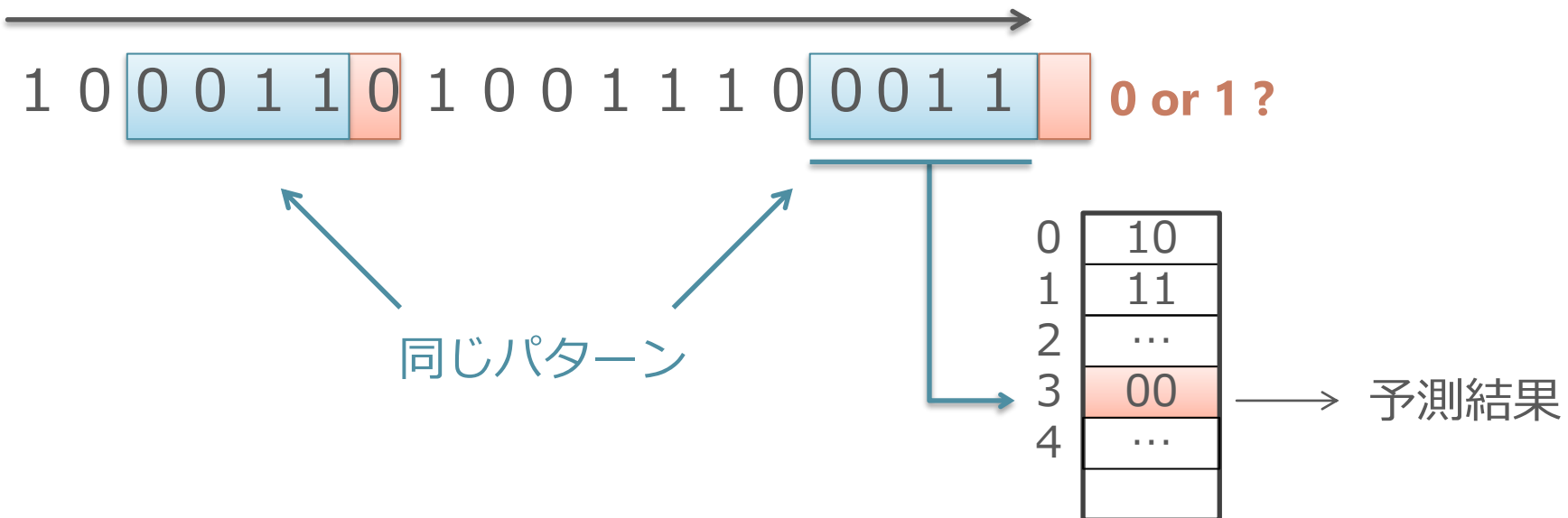
```
for (i = 0; i < 4; i++) {  
    ...  
}
```

- 動的に分岐の方向が頻繁に変わるものに対応できない
- 例：4回まわる for ループ
 - 4 回のうち 3 回は成立，1 回が不成立となる
 - TTTFTTTFTTTFTTT...
 - 2 ビット・カウンタ予測器の精度は $3/4 = 75\%$ に
- モチベーション：
 - しかし明らかに規則性があるので，なんとか予測できないか？

「履歴（history）」を用いた予測器

- 基本的なアイデア：分岐方向の履歴をビット列で表す
 - 履歴のビット列をインデクスとしてテーブルにアクセス
 - テーブル自体は、2 ビットカウンタ
 - 直前の履歴でテーブルをひく
 - 直前に同じパターンがくると、同じエントリにアクセス
 - 二進数で 0011 = 表の3番目のエントリ
 - より一般化すると、履歴をハッシュ値にし方向とともに憶える（後述）

履歴 成立：1 不成立：0



履歴とエントリの対応

PHT (2ビット・カウンタ)

直前の履歴	0000	00
	0001	11
	0010	11
	...	...
	1010	00
		...

- 直前の履歴ごとに、異なる PHT のエントリが割り当てられる
 - 0001 : カウンタが 11 なので、このパターンは次は 1
 - 1010 : カウンタが 00 なので、このパターンは次は 0

PHT アクセス時のインデクスの生成は、履歴と PC を混ぜる

- 全てのアドレスの分岐でエントリが共有されてしまう
 - たとえば直前の履歴が同じ 1010 場合,
PC が 0x4000 の分岐も 0x8044 の分岐も同じエントリを使ってしまう
 - (0x4000 とかは適当で、意味はない)
- 対策の例：PC と履歴をビット結合する
 - 0x4000 と 0xa (2進で1010=0xa) を結合 → 0x4000a をインデクスに
 - 0x8044 と 0xa (2進で1010=0xa) を結合 → 0x8044a をインデクスに

履歴を用いた予測器

- この履歴の保持方法/作り方の違いで、いくつかの方法がある

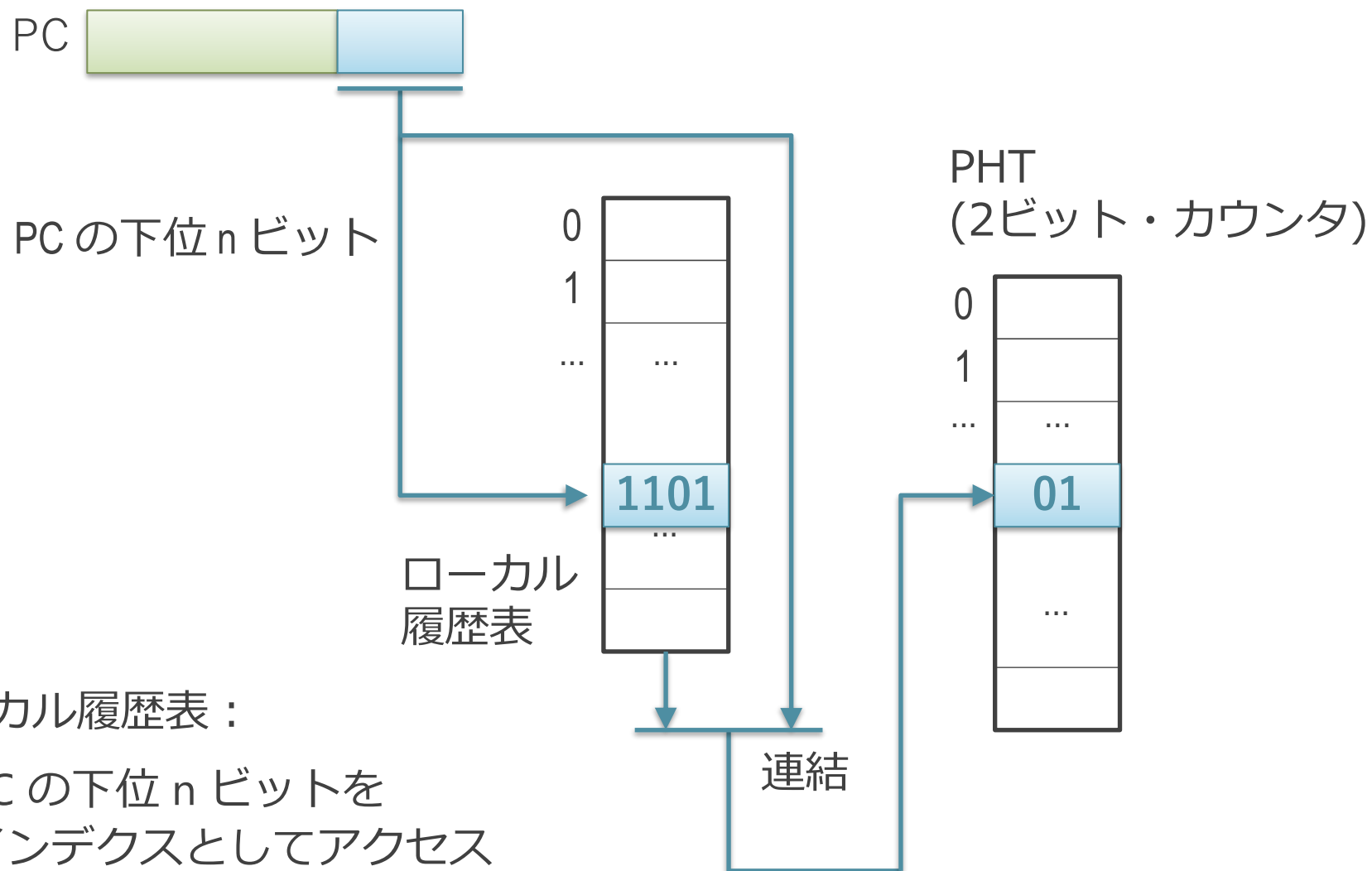
1. ローカル履歴予測器

- PC ごとに履歴を保持

2. グローバル履歴予測器

- PC を区別せず履歴を保持

ローカル履歴予測器

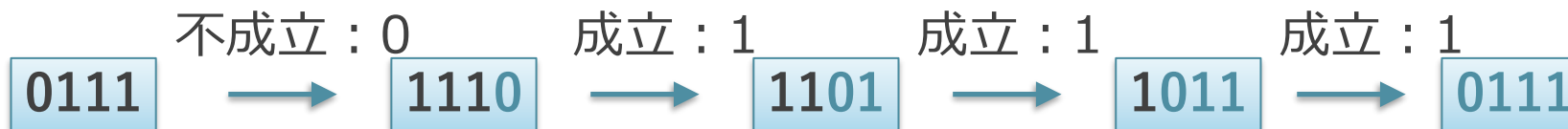


■ ローカル履歴表：

- PC の下位 n ビットをインデクスとしてアクセス
- 各エントリは複数ビットのシフト・レジスタ

ローカル履歴表

- その PC の分岐の, 過去に分岐方向のパターンを表す
- 分岐が実行されるごとに,
 - 全体を左にシフトし
 - 右側から新しい結果を挿入
- 下の図の場合は 4 回に 1 回, 不成立 0 が挿入されている
 - 4 回だけ回る for ループのパターン



ローカル履歴予測器の動作例（1）

```
for (i = 0; i < 4; i++) {  
    ...  
}
```

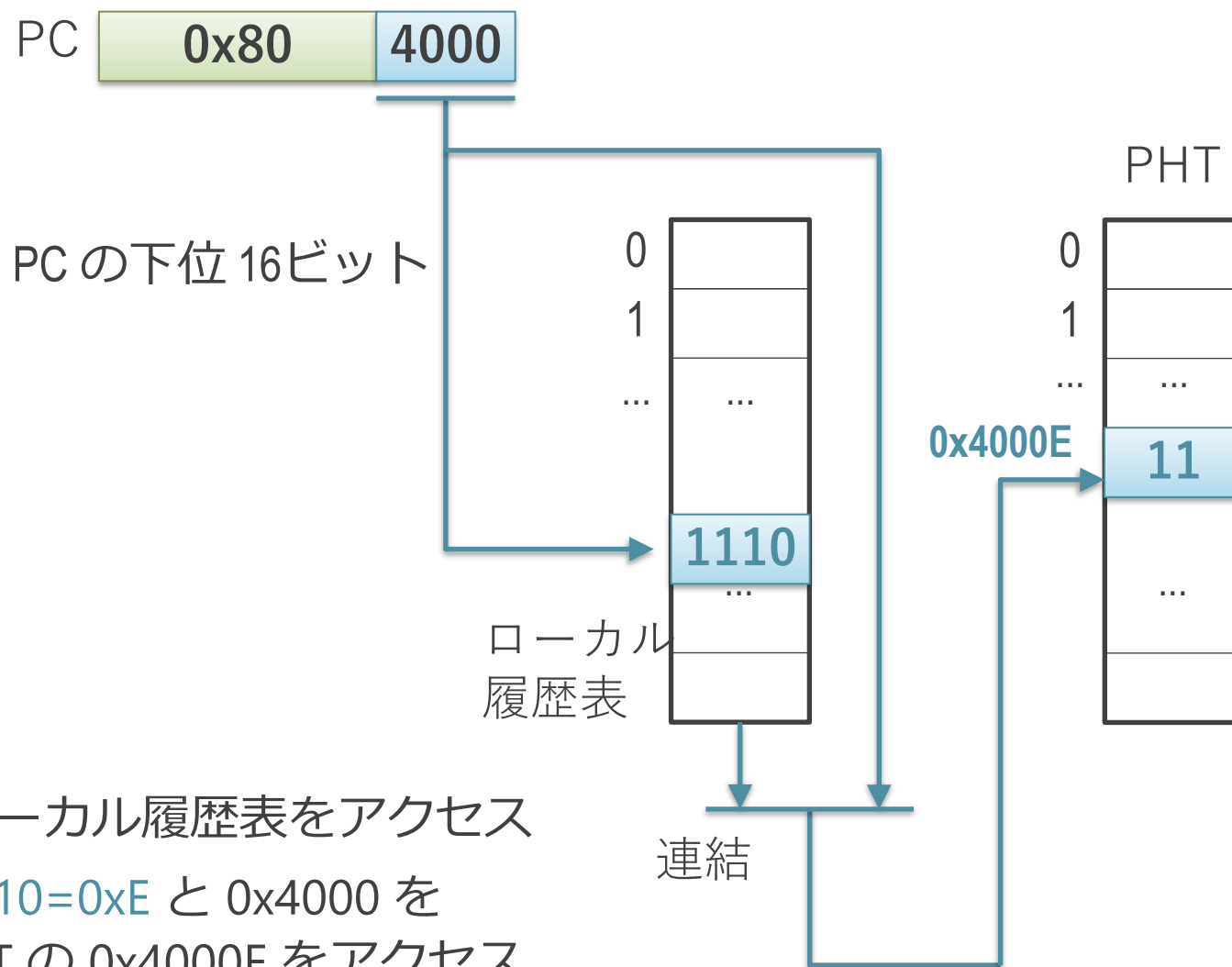
// 4回まわるループ

```
0x803ff8:      li  x1 ← 0           // x1 を 0 に初期化  
      L:  
0x803ffc:      add x1 ← x1 + 1     // x1 をインクリメント  
0x804000:      bne x1 != 4, L      // x1 が 4 でなければ L に飛ぶ
```

■ 例：4回まわるループ

- 4回のうち3回は成立，1回が不成立
- このループの後方分岐が 0x804000 にあったとする

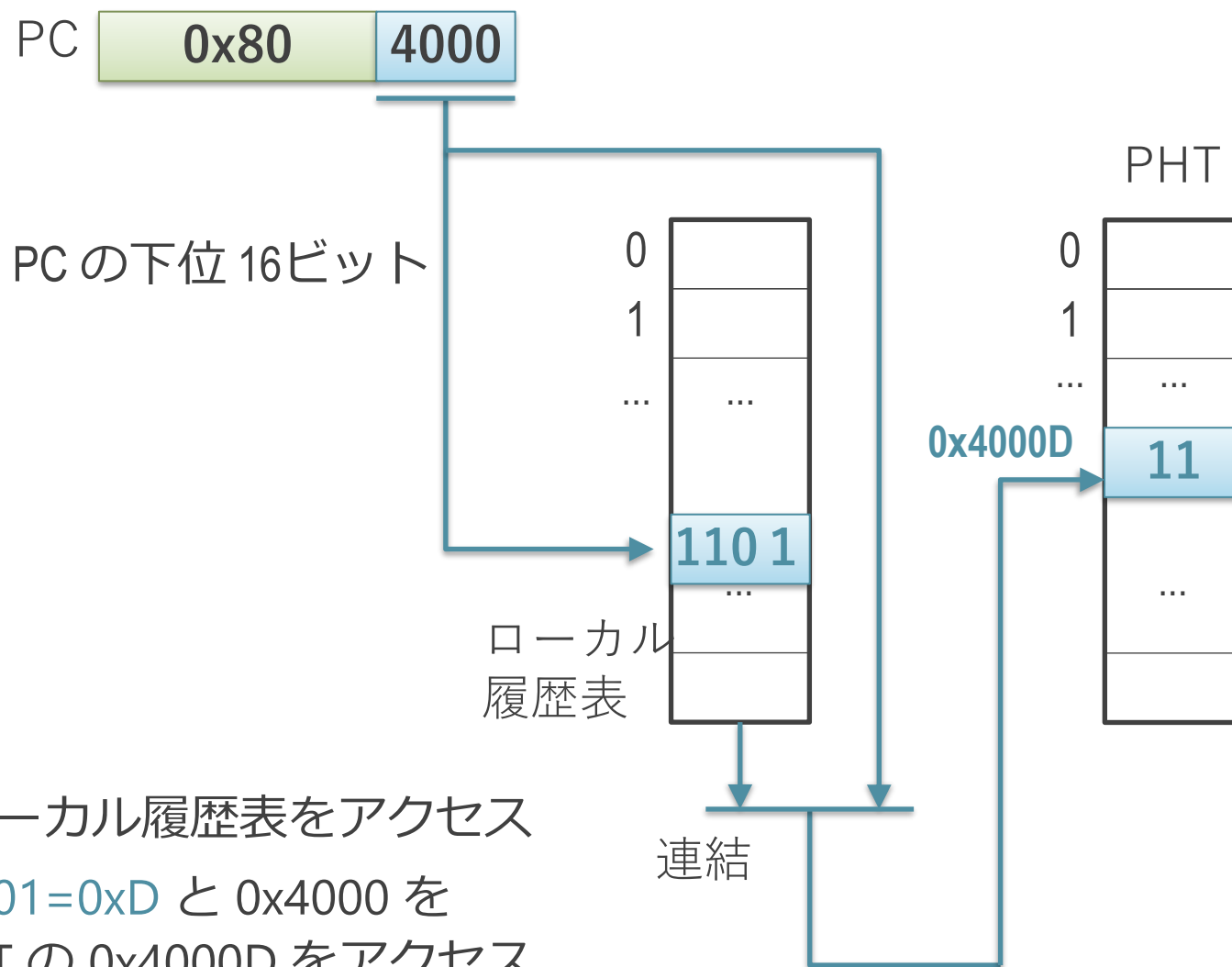
ローカル履歴予測器の動作例（２）



■ 動作：

1. 0x4000 でローカル履歴表をアクセス
2. 得られた **1110=0xE** と 0x4000 を結合し, PHT の 0x4000E をアクセス
3. カウンタの中身が **11** なので成立と予測

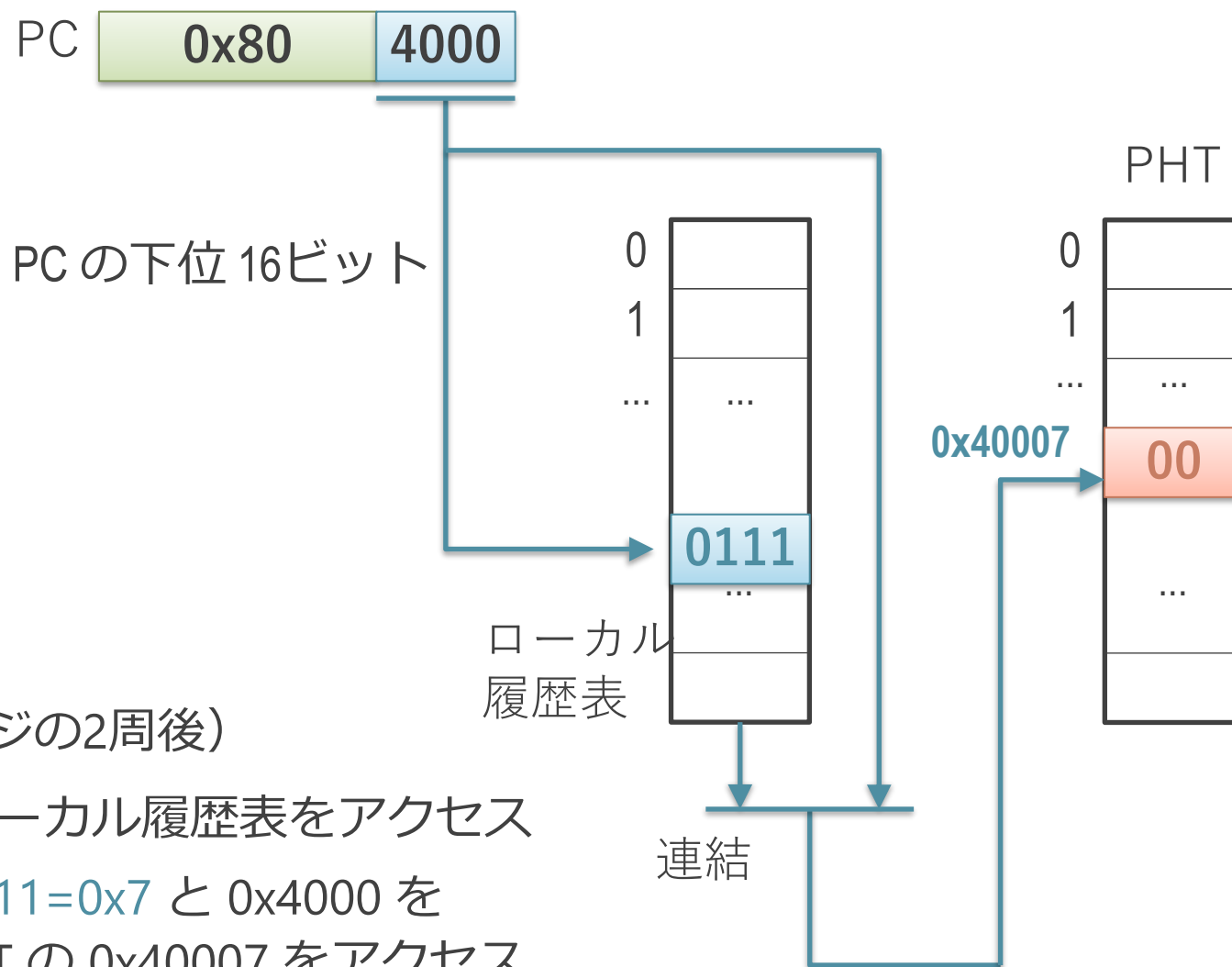
ローカル履歴予測器の動作例（3）



■ 動作：

1. 0x4000 でローカル履歴表をアクセス
2. 得られた 1101=0xD と 0x4000 を結合し, PHT の 0x4000D をアクセス
3. カウンタの中身が 11 なので成立と予測

ローカル履歴予測器の動作例（3）



■ 動作：（前ページの2周後）

1. `0x4000` でローカル履歴表をアクセス
2. 得られた `0111=0x7` と `0x4000` を結合し, PHT の `0x40007` をアクセス
3. カウンタの中身が `00` なので不成立と予測

ローカル履歴予測器のメリット

- 特定の PC の分岐方向にパターンがある場合, 有効に働く
- たとえば,
 - 成立と不成立を交互に繰り返す
 - 短い for ループ

履歴を用いた予測器

- この履歴の保持方法/作り方の違いで、いくつかの方法がある

1. ローカル履歴予測器

- PC ごとに履歴を保持

2. グローバル履歴予測器

- PC を区別せず履歴を保持

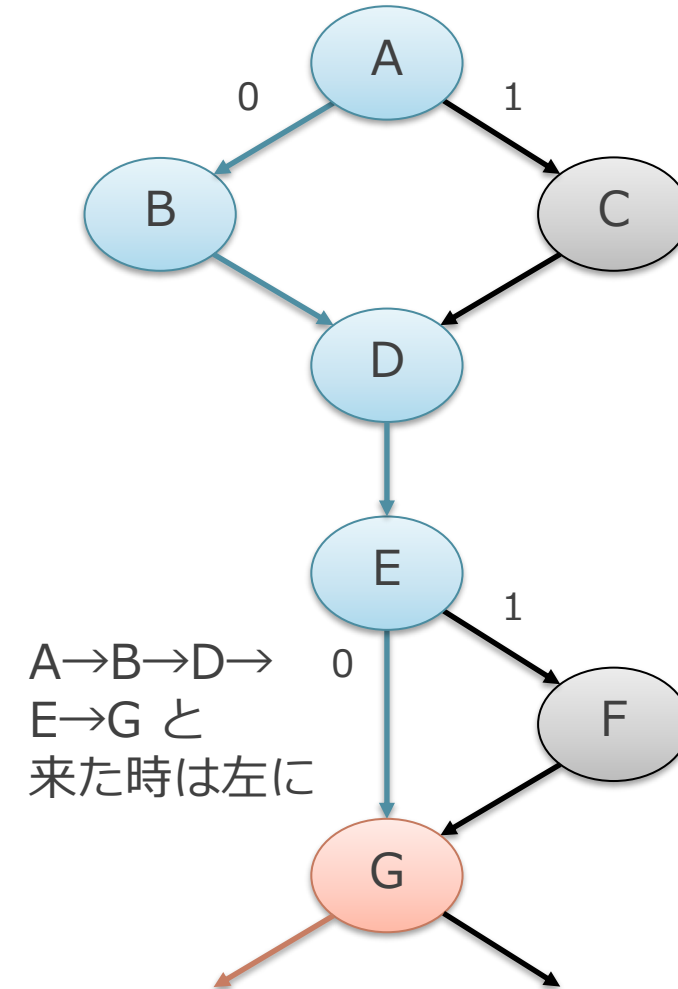
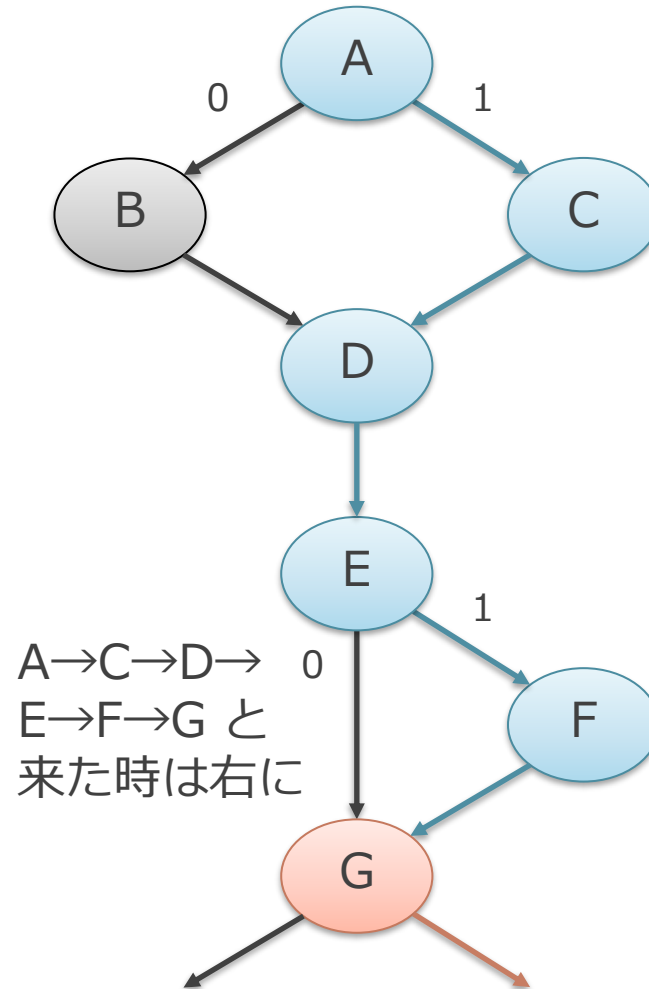
グローバル履歴予測器のモチベーション

```
if (option == 1) {  
    ...  
}  
  
...  
if (option != 1) { // 上の if と必ず反対になる  
    ...  
}
```

- 複数の分岐間に相関があることが結構多い
 - ある分岐が成立したら, その後ろにある別の分岐は不成立... など
- こう言う相関を拾いたい

グローバル履歴予測器のイメージ

```
if (A) {  
    B...  
}  
else{  
    C...  
}  
D...  
if (E) {  
    F...  
}  
if (G) {
```



- 「こういうパスを通ってきたときは、成立 or 不成立になりやすい」をうまく予測したい

ローカル履歴予測器とグローバル履歴予測器

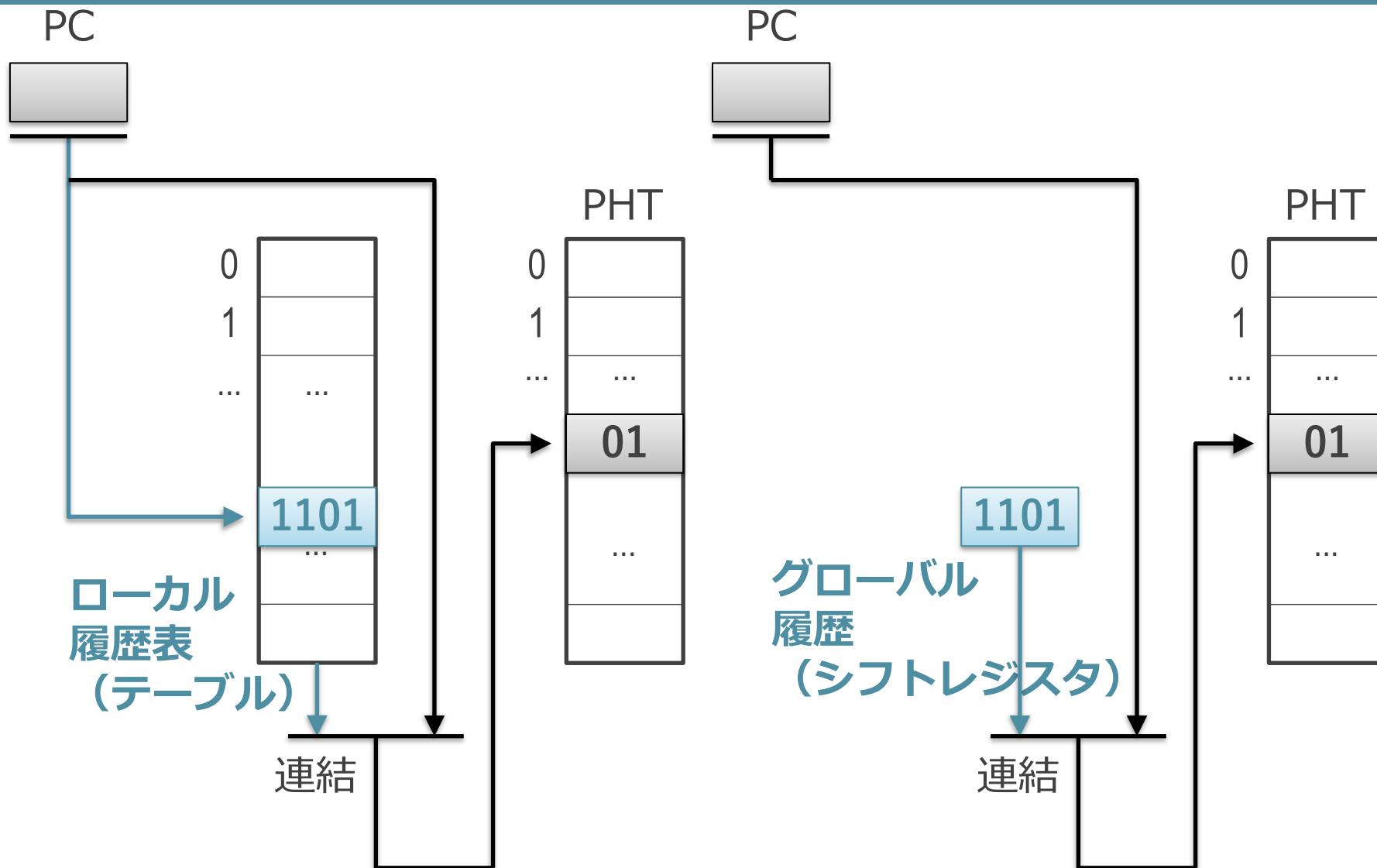
■ ローカル履歴予測器

- 各静的分岐のアドレスごと（ローカル）に，分岐方向の履歴を保持

■ グローバル履歴予測器

- 各静的分岐を区別せず（グローバル）に，分岐方向の履歴を保持
- 直前に実行した分岐の方向をどんどん保存していく
- あとはローカル履歴予測器と同じ

ローカル履歴予測器とグローバル履歴予測器



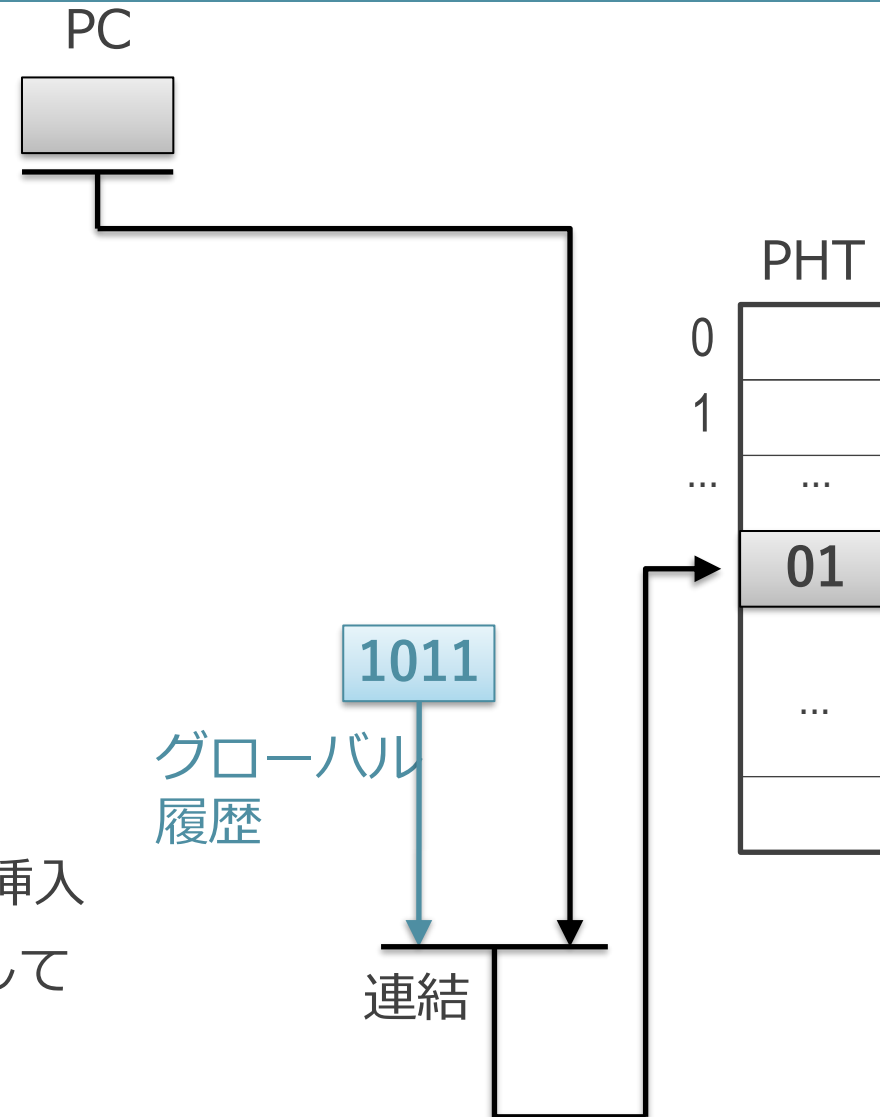
- 1 エントリのローカル履歴表を全員で共有しているイメージ

グローバル履歴予測器

```
if (option == 1) {} // 1
if (option != 1) {} // 0
if (...) {} // 1
if (...) {} // 1
if (...) {} // ???
```

■ 動作：

- 分岐命令が実行されるたびに、分岐方向をグローバル履歴に挿入
- グローバル履歴と PC を連結して PHT にアクセスし、予測



グローバル予測器の利点

■ 利点：

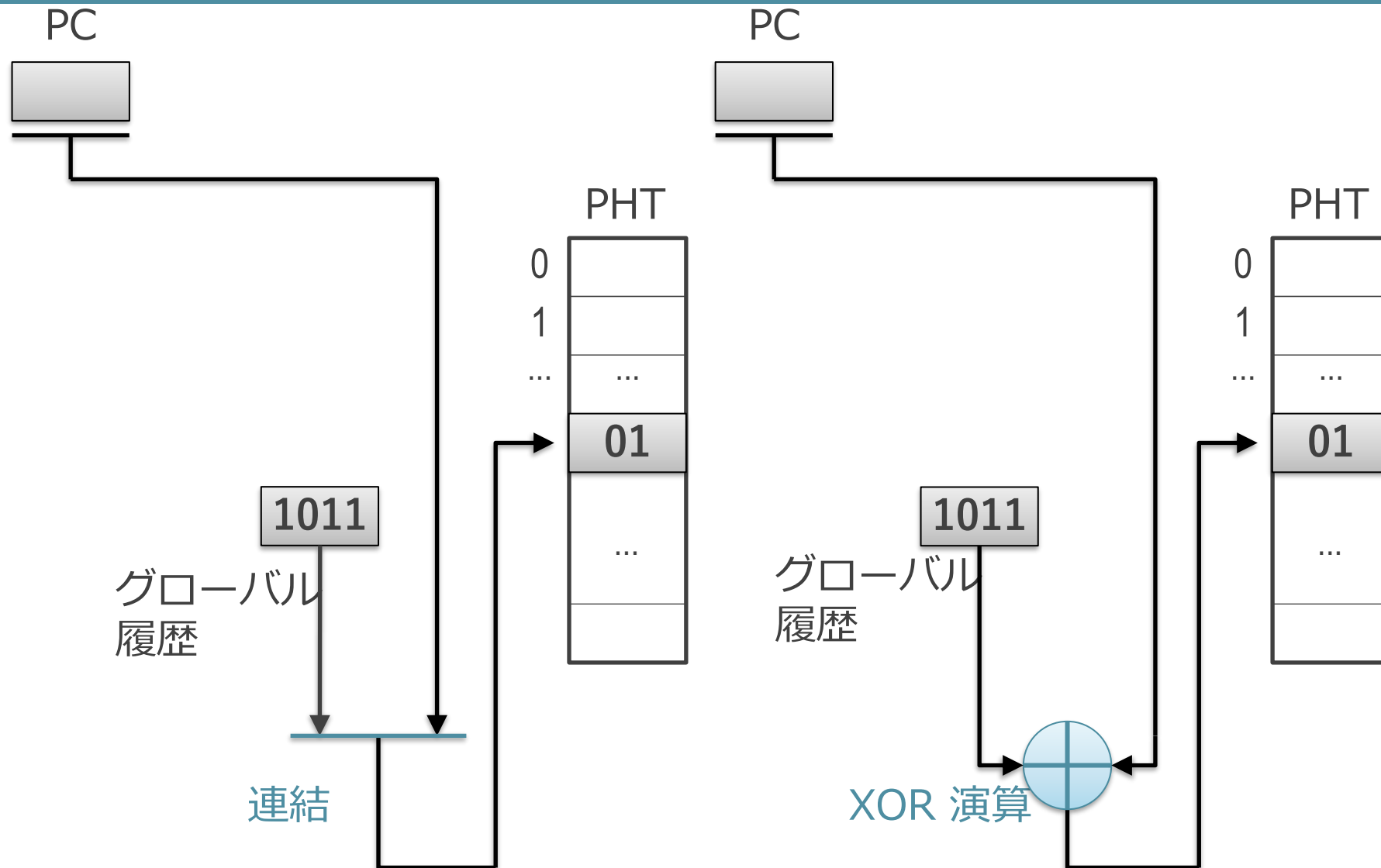
1. 異なる静的分岐の間にある相関を拾える
 - 例：～行目の if と ～ 行目の if は常に同じ方向
 - ローカル履歴予測器では拾えない
2. ローカル履歴に対応する相関も拾える
 - 直前に実行された動的分岐の方向を区別なく使用
 - なのでループのような同じアドレスの分岐も内包している

履歴長と予測精度

- 一般に、グローバル履歴長を長くするほど精度はあがる
 - より遠い分岐の相関が拾えるようになる
 - 履歴長が1000以上のところに相関がある場合もある
 - ある関数で分岐した後、色んな所にいったてまた来るとか
- 実際にはハードウェア（特に PHT の大きさ）の制約がある
 - 1 サイクル内にアクセス可能な大きさに限られる
 - 最大数K エントリ程度
 - （最近はもうちょっと大きいかも）
 - 履歴長に対し、2 の累乗のオーダーでエントリ数が増加

- グローバル履歴予測器の一種
 - より長い履歴長でも, エントリ数が大きくならないようにしたもの
- モチベーション: グローバル履歴のパターン自体に偏りがある
 - たとえば履歴長が16ビットだとして, 2の16乗の全てのパターンは通常現れない
 - 単純に PC と連結すると, 使われないエントリの方が圧倒的に多い
- ビット連結ではなく, XOR 演算により結合
 - ビット連結: PC 下位16ビット + 履歴16ビット → 32ビット
 - XOR 演算: PC 下位16ビット + 履歴16ビット → 16ビット

g-share 予測器



- ビットを単純に連結するかわりに, XOR 演算して結合

なぜ XOR 演算なのか？

AND

<i>a</i>	<i>b</i>	<i>z</i>
0	0	0
0	1	0
1	0	0
1	1	1

OR

<i>a</i>	<i>b</i>	<i>z</i>
0	0	0
0	1	1
1	0	1
1	1	1

XOR

<i>a</i>	<i>b</i>	<i>z</i>
0	0	0
0	1	1
1	0	1
1	1	0

■ 要求：

- 軽量の演算であること → 論理演算が良い
- 2つの値がバイアスせずに混じってくれること
(0と1が均等に現れること)

■ 要求を満たす論理演算は、XOR か XNOR しかない

- AND や OR では、結果が0か1に偏る
- それ以外は、*a* か *b* そのものか、それらの反転になってしまう

パーセプトロンと TAGE

より高度な予測器

1. パーセプトロン予測器
2. TAGE 予測器

- パーセプトロンと TAGE は基本的にはグローバル予測器が下敷き
 - XOR 演算は要素としてよく出てくる

予測器の精度

- 左上が g-share, 右下が比較的新しい TAGE

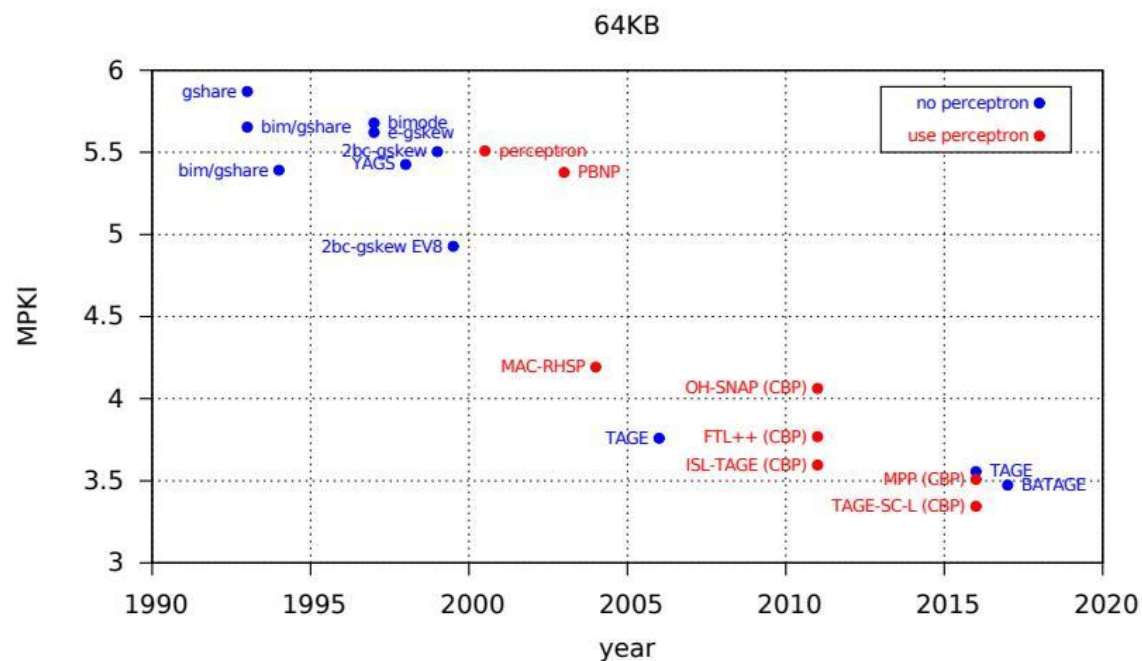


Figure 1: Average number of mispredictions per 1000 instructions (MPKI) for various conditional branch predictors on the CBP 2016 traces for 8KB, 32KB and 64KB storage budgets (see Appendix A).

高度な予測器

- パーセプトロン予測器
- TAGE 予測器
- これらのより詳細はこの講義では扱いませんが、高度な分岐予測器の詳細に関する付録の資料を用意しています
- github のリポジトリの末尾の方にあります

- 一般に、グローバル履歴長を長くするほど精度はあがる
 - より遠い分岐の相関が拾えるようになる
 - 履歴長が1000以上のところに相関がある場合もある
 - ある関数で分岐した後、色んな所にいったてまた来るとか

履歴長と予測精度

- 実際にはハードウェア（特に PHT の大きさ）の制約がある
 - 数サイクル内にアクセス可能な大きさに限られる
 - 単純にやると履歴長に対し、2 の累乗のオーダーでエントリ数が増加
 - インデクスの最大値 = $2^{\text{履歴長}} - 1$
 - ハッシュ関数で折りたためば解決するが、出現パターン数が増えることには違いがない

パーセプトロン予測器

■ モチベーション：

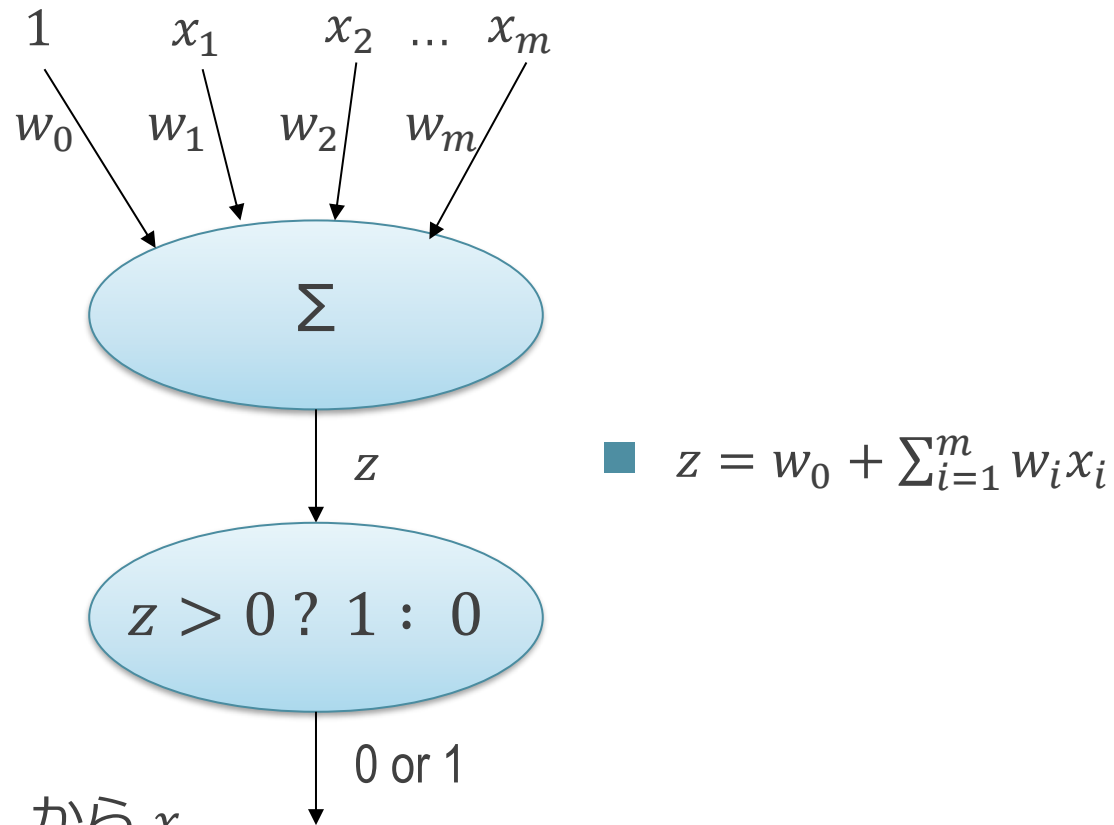
- グローバル履歴のうち、本当に相関があるのは一部のビットのみ
- ある特定の if 文同士で相関がある場合、間の履歴は無駄

■ パーセプトロン予測器：

- 1 層パーセプトロンを使って予測
- 高速に予測を行う必要があるため、ややこしいことは無理
 - 1 層限定で、重みは 8 ビット固定小数点とか

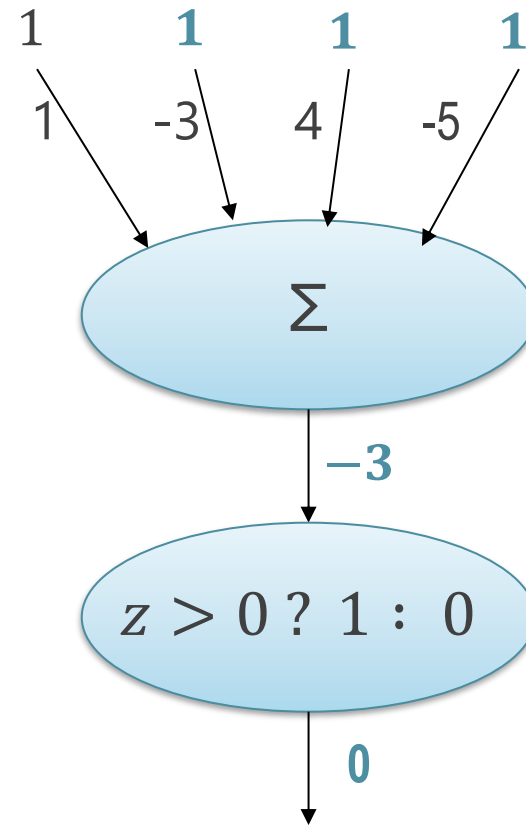
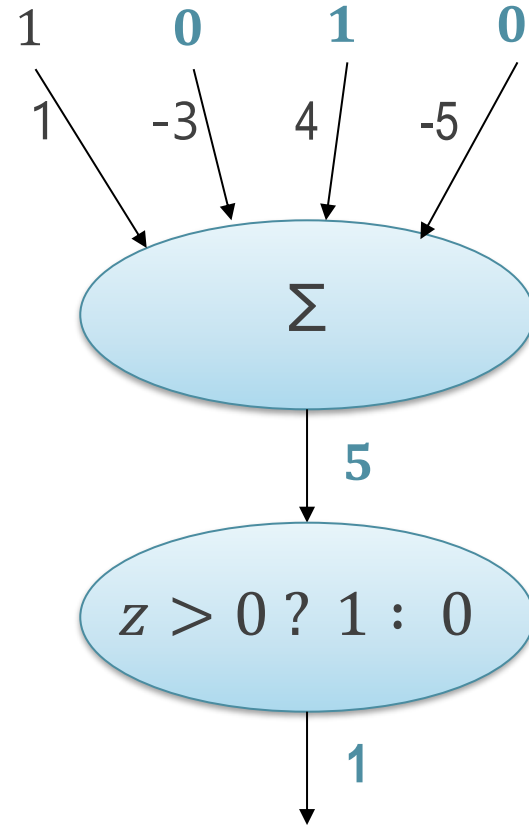
■ 塩谷が学生の頃は半分ネタだと思われていたが、今は普通に使われている

パーセプトロン



- ◇ 入力 (0 or 1) : x_1 から x_m
- ◇ 重み (アナログ値) : w_0 から w_m (w_0 はバイアス)
- ◇ 学習 :
 - z が 1 の時, x_i が 1 だったなら w_i を大きく
0 だったなら w_i を小さくする

「010 なら 1」を学習させた場合

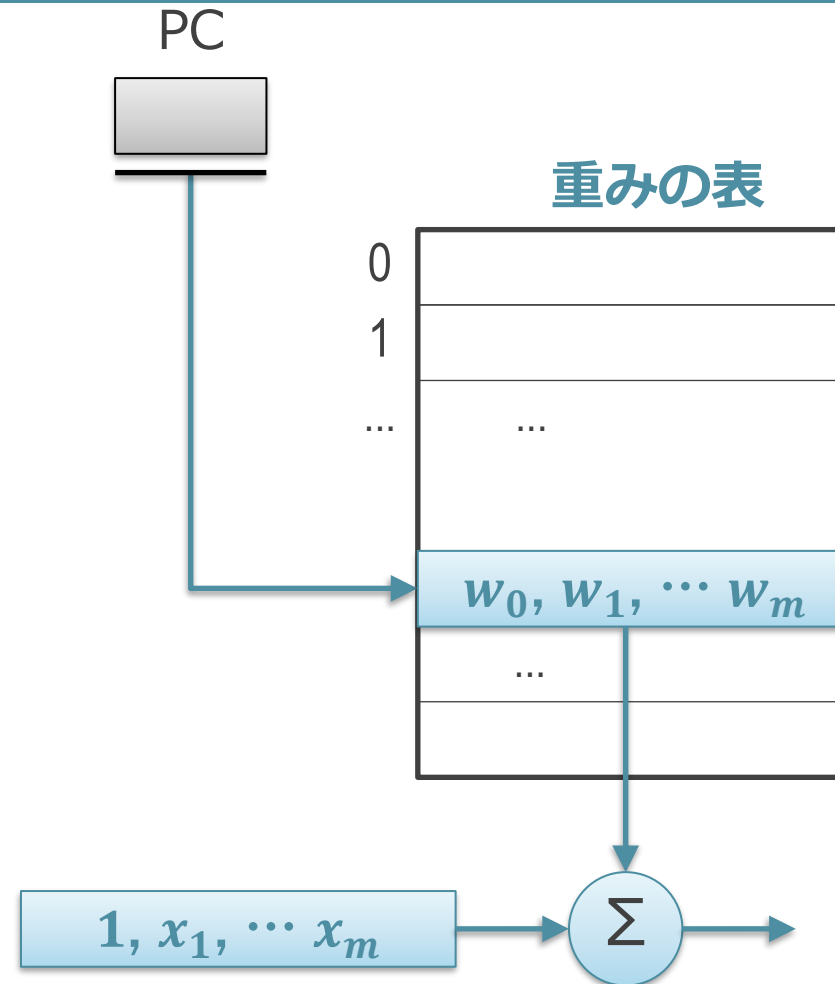


- 010 以外のパターンが来た場合は, z は負の方向に
- 左端の係数はバイアス
 - 真ん中の「 $z > 0 ?$ 」は, 「 $z < w_0 ?$ 」と等価の意味になる

■ バリエーション

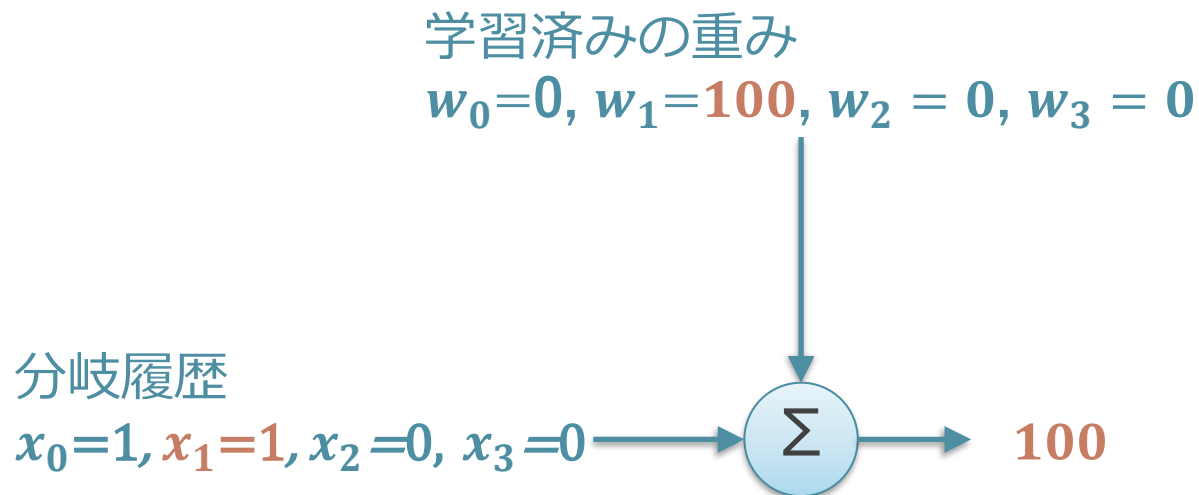
- Liner perceptron 系 :
 - グローバル履歴をそのまま重みにかけるもの
- Hashed perceptron 系 :
 - g-share 的なテーブルから重みを出すもの

グローバル履歴をそのまま重みにかけるもの



- PCの一部をインデクスとして重み表をひく
 - そのPCに対応した重みのセットがとれる
- あとはグローバル履歴を x_i としてパーセプトロンの処理を行う

実際の学習の様子



■ x_4 の方向を予測する

- x_4 は x_2 x_3 の方向とは関係なく, x_1 の方向のみと相関する場合を考える
- x_1 に対応する w_1 の重みを大きく, それ以外の絶対値を小さく
なるよう学習すればよい
 - 分岐方向に応じて毎回重みを加算/減算すれば,
成立に偏る場合は大きくなり, 無相関な場合は 0 に近づく
- 内積をとると, x_1 が 0 か 1 かにのみ応じて結果が変わる

Liner perceptron 系

1. 最初に提案されたパーセプトロン予測器

Jimenez, Daniel A et al., Dynamic branch prediction with perceptrons, HPCA 2001

2. Path-Based 予測器

Jimenez, Daniel A et al., Fast Path-Based Neural Branch Prediction, MICRO 2003

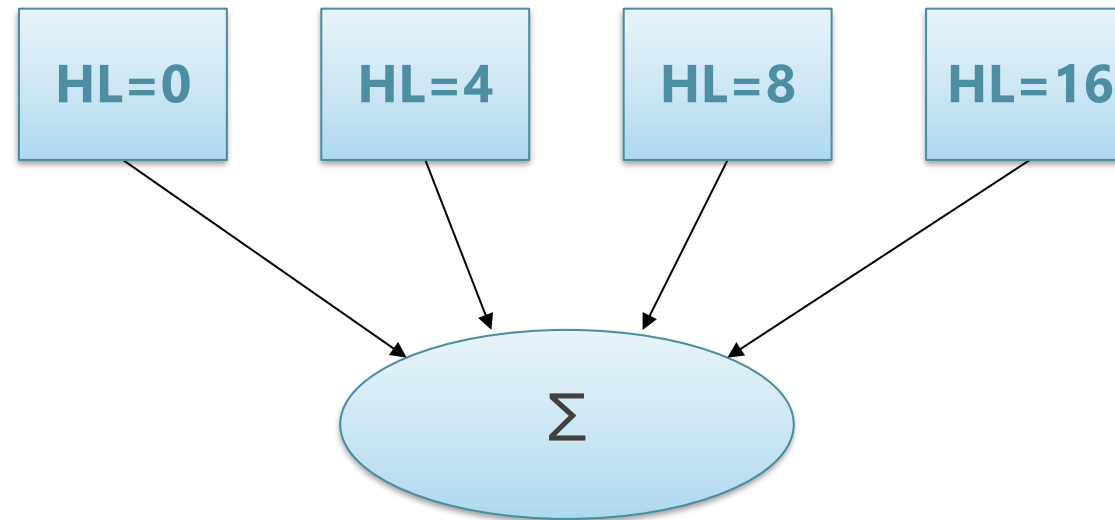
3. Piecewise Linear 予測器

Jimenez, Daniel A et al., Piecewise Linear Branch Prediction, ISCA 2005

富士通の A64fx にはこれが搭載されている

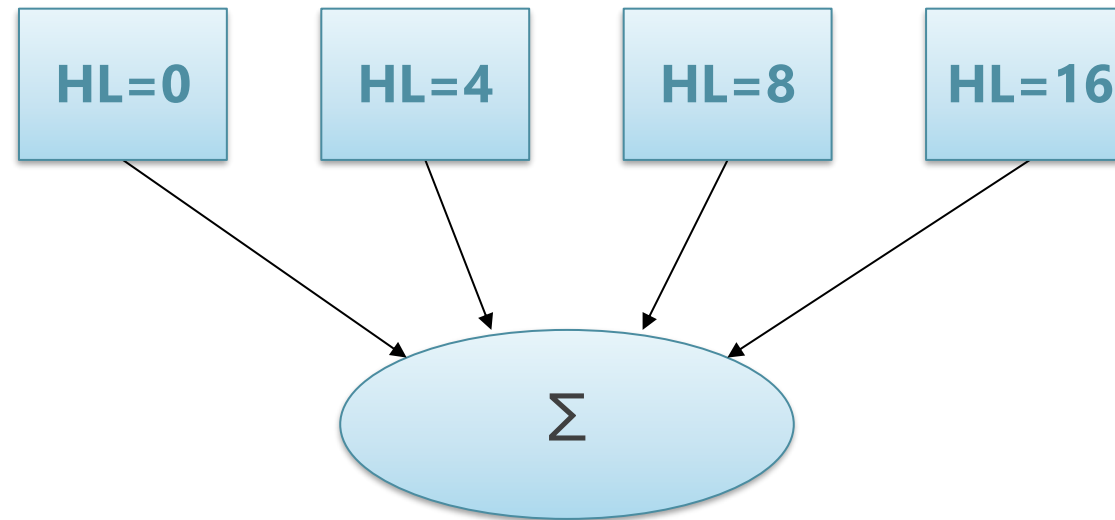
- 「付録：分岐予測の詳細」にて、これらは詳細を説明

Hashed perceptron



- 履歴長 (HL) が異なるグローバル予測器のようなテーブルを複数用意
 - 履歴 と PC の組み合わせでテーブルを引く (グローバル予測器と同じ)
 - 中身は 2 ビットカウンタではなく, 重み (8 ビットなど) にする
- History length (HL)=4 のテーブルの場合
 - 4ビットの履歴と PC を XOR してそれをインデックスにしてアクセス
 - 8 ビットの重み (符号付き) が取れてくる
 - 符号なし 2 ビットにすれば本当にただのグローバル分岐予測器と等価

Hashed perceptron



- 全員の出力を加算してパーセプトロンの処理
 - 通常のパーセプトロン予測器と違い, 各テーブルから出てきた重みはそのまま加算される
 - 重みつき多数決のイメージ
 - 履歴と重みの内積はとらない
 - 履歴は各テーブルをアクセスするときに使用されている

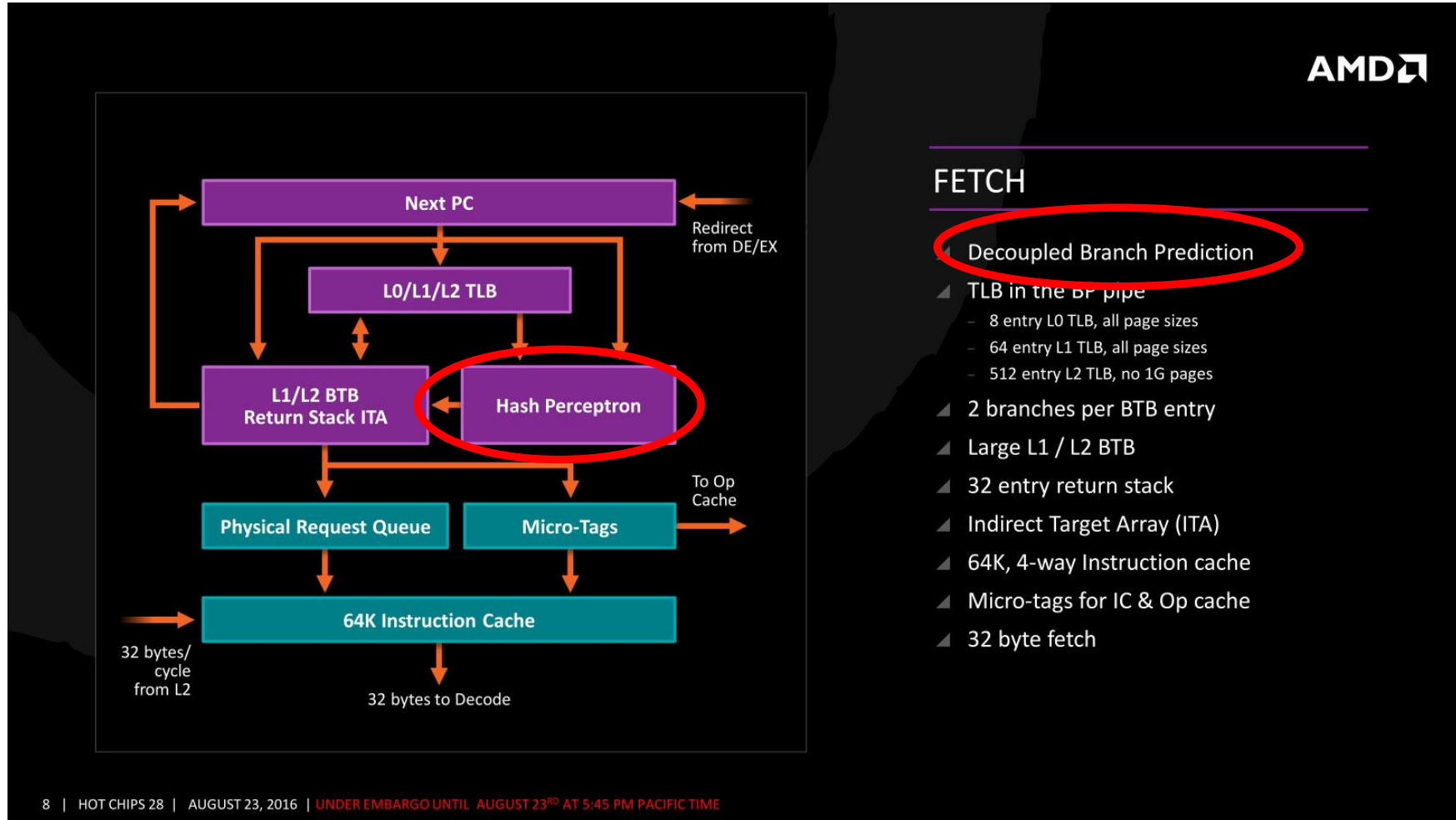
Hashed perceptron の効果

- g-share や linear perceptron 予測器の問題
 - 長い履歴を捉えようとするすると履歴長に比例した記憶容量が必要
 - 長い履歴を憶えようとするすると、短い履歴で済む場合も大きな記憶容量が必要
- ハッシュ・パーセプトロンの効果：
 - 長い履歴をきちんと憶えつつ、履歴が短い大半の場合は少ないエントリ数で記憶できる
 - 相関がある履歴長のテーブルの重みの絶対値が増えるように学習される
 - （正確には、そこより長い履歴長のテーブル

ハッシュ・パーセプトロンの効果の例

- たとえば 101 と 11111 の2パターンがあった場合
(左から右に taken:1, untaken:0)
- 固定長グローバル履歴予測 (HL=5)
 - 5カ所を更新 : 00101, 01101, 10101, 11101, 11111
- ハッシュ・パーセプトロン (HL=3,5) :
 - 101 : 主に1カ所を更新
 - HL=3のテーブルでは, 101 の重みを大きく
 - HL=5のテーブルでは, 00101, 01101, 10101, 11101 に散らされる
 - 11111 : 主に1 or 2カ所を更新
 - HL=3のテーブルでは, 111 の重みを大きく
 - HL=5のテーブルでは, 11111 の重みを大きく

AMD Zen では Hashed Perceptron を使ってる



TAGE 予測器

■ TAGE 予測器

- A. Seznec and P. Michaud. A case for (partially)-tagged geometric history length predictors, Journal of Instruction Level Parallelism (<http://www.jilp.org/vol8>), 2006

■ 現在最も予測精度が高いと言われている予測器

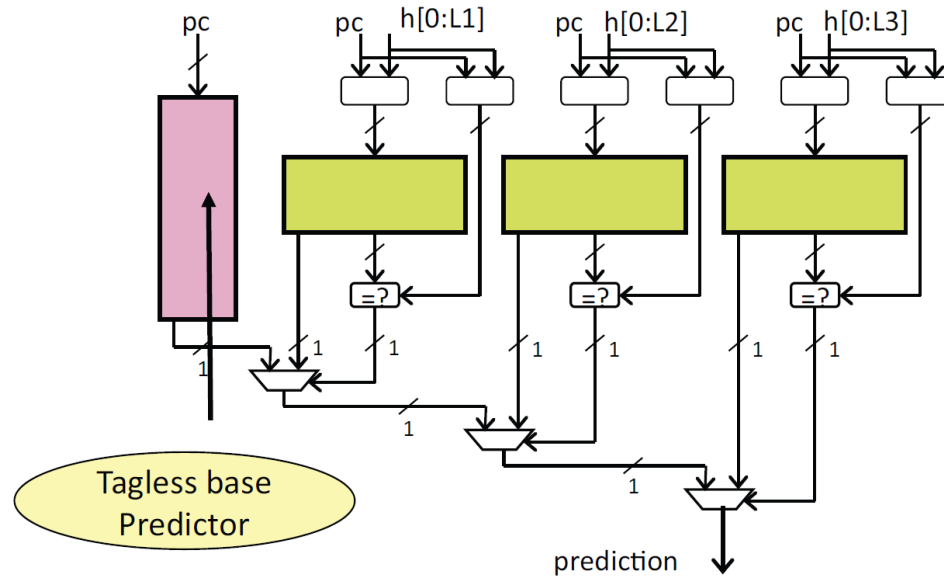
- 2026 年現在の高性能 CPU は知っている限り全部これベース
 - Intel, AMD, ARM, Apple...
- 多くの論文でリバースエンジニアリングされている

■ モチベーション

- 必要となるテーブルのエントリ数は2の履歴長乗に比例するので, なるべく履歴が短いテーブルだけで学習を済ませたい

TAGE 予測器

図は A. Seznec and P. Michaud.: A case for (partially)-tagged geometric history length predictors より

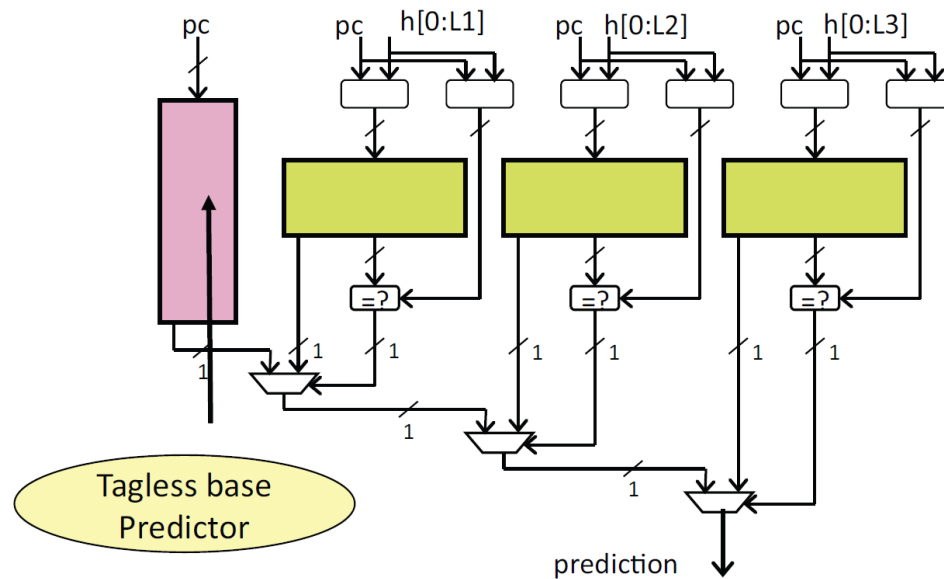


■ トーナメント式の構造：

- 左端： 単純な 2 ビット・カウンタ
- それ以外：
 - グローバル履歴+PC でアクセスし, BTB の時のように ヒット/ミス判定を行う
 - ヒットした場合, そのテーブルの n ビット・カウンタの内容で予測

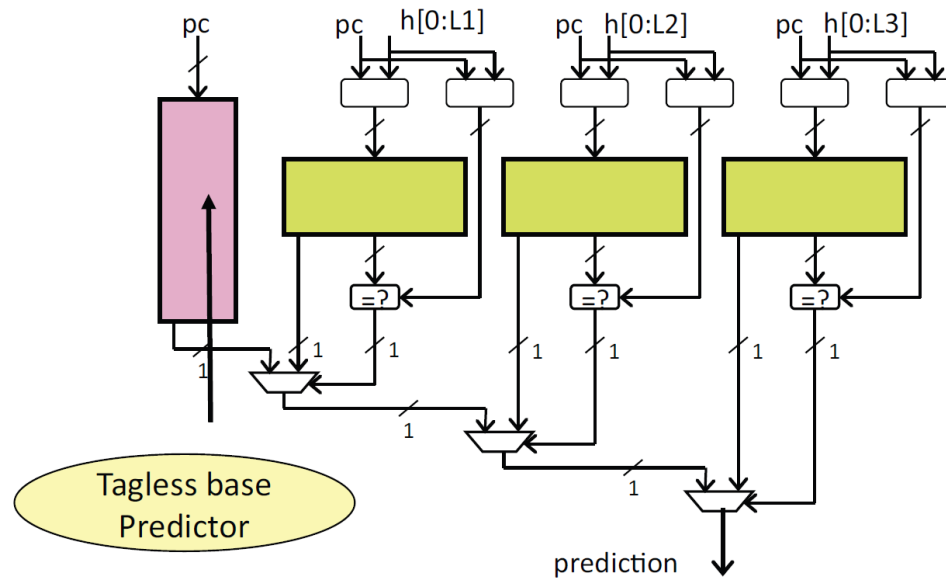
TAGE 予測器

図は A. Seznec and P. Michaud. A case for (partially)-tagged geometric history length predictors より



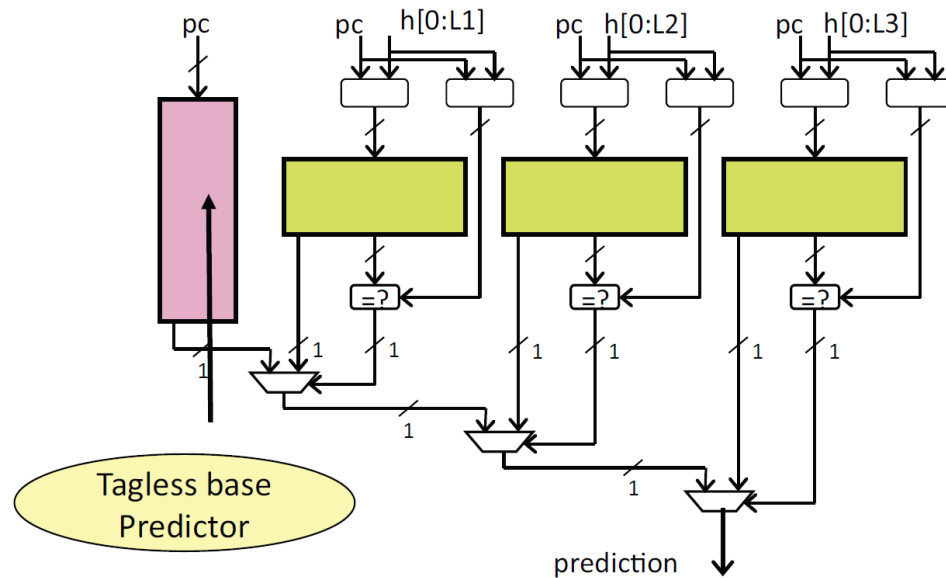
- パターン長が長いテーブルの結果を優先して使う
 - 右側でヒットするほど，その結果を優先する
 - 右に行くほど指数的に履歴長が長くなっている
- ヒット/ミスの判定ができるため，トーナメント状に優先度が決定できる
 - ただの PHT だと，だれの結果を使えばいいかわからない

TAGE 予測器のメリット（１）



- 利点 1 : パターン長ごとに, 最適なテーブルに学習できる
 - 記憶すべきパターン数が減る
 - たとえば 101 と 11111 の2パターンがあった場合
 - 固定長(5) : 00101, 01101, 10101, 11101, 11111
 - TAGE(3+5) : 101, 11111

TAGE 予測器のメリット（２）



■ 利点 2 : 学習が早い

- 大ざっぱに成立/不成立な傾向があるような場合,
- 難しいパターンを学習する前に, とりあえず左端で大ざっぱな予測ができる
 - 左端は履歴を使わないただのカウンタだから

TAGE 予測器 と ハッシュパーセプトロン予測器

- それぞれ、今一番予測精度が高いと思われる
 - インテルや AMD の CPU でも実際に採用
- TAGE の方が、基本的には良い精度を示す
 - タグでドンピシャで憶えるほうが、重みが散るより容量効率がよい

予測器の精度

Pierre Michaud, An alternative TAGE-like conditional branch predictor, TACO 2018 より

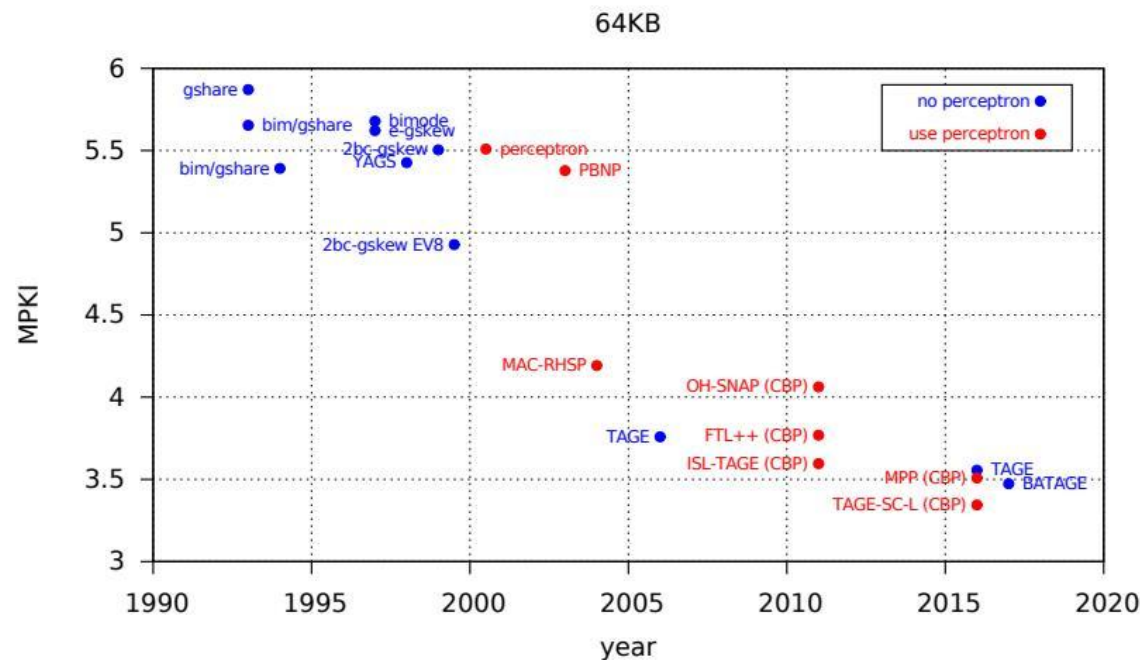


Figure 1: Average number of mispredictions per 1000 instructions (MPKI) for various conditional branch predictors on the CBP 2016 traces for 8KB, 32KB and 64KB storage budgets (see Appendix A).

■ TAGE-SC-L と呼ぶある種のハイブリッド型が強い

- statistical corrector (SC) は, 一種のハッシュパーセプトロン
- TAGE の結果を補正する

分岐予測のアルゴリズムの発展

- 基本的には André Seznec 先生が作った TAGE-SC が最も予測精度が高いと言われている
 - 写真 は <https://team.inria.fr/pacap/members/andre-seznec/> より
- Championship Branch Prediction
 - 分岐予測は, この大会で出た様々なアイデアを元に発展してきた
 - Seznec 先生がもうずーっと優勝し続けていた (20年以上)
 - 2025 年に久しぶりに開催



CBP2025 Opening Remarks

Sixth Championship Branch Prediction (CBP)

Co-located with ISCA 2025 – Tokyo, Japan

Rami Sheikh
Organizing Committee Chair

June 21, 2025



arm

去年はうちが優勝

CBP2025 Winners ...

Nagoya Institute of Technology
The University of Tokyo

CBP2025 Winner



1st Place \$ prize: \$1,000



arm



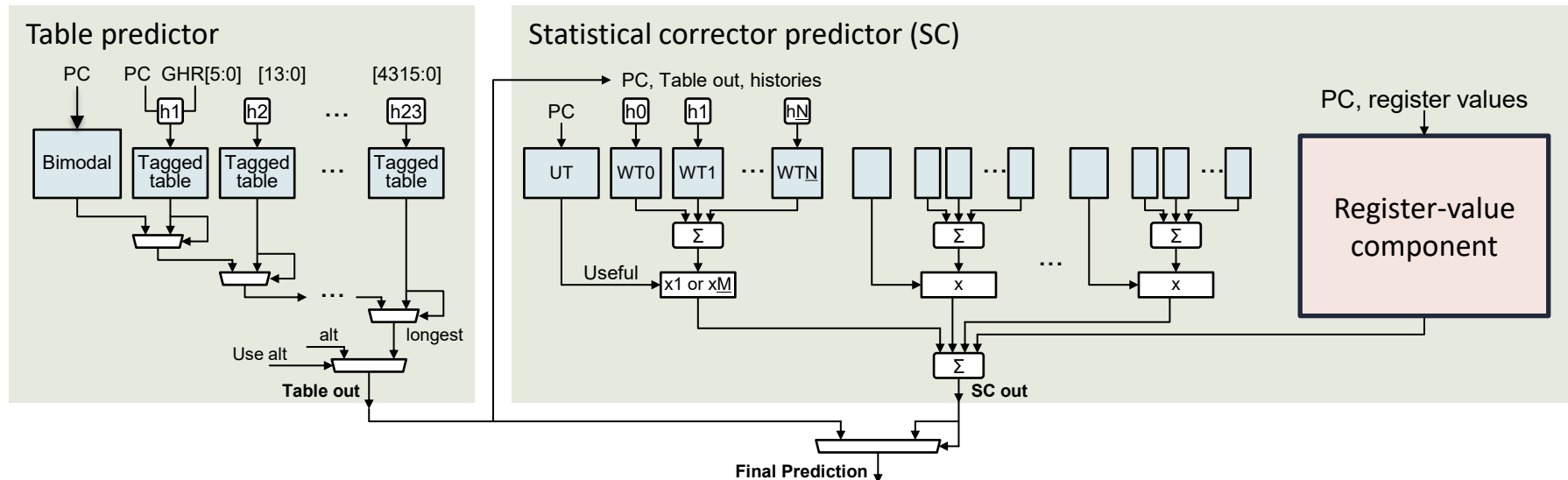
RUNLTS 予測器

世界でもっとも強い分岐予測器の1つ（2026年5月現在）

■ 基本的には TAGE-SC 予測器の構造を踏襲

1. エントリのアロケーションポリシー
2. ヒストリ長の取り方を幾何級数から異なるものに
3. SC をいろいろ改良
4. RBias : レジスタ値と分岐方向の相関を捉える新しい機構

■ Toru Koizumi et al., "RUNLTS: Branch Prediction with Register-Value Correlations and Hierarchical Table Orchestration", ISCA 2026 でフルペーパーの発表予定

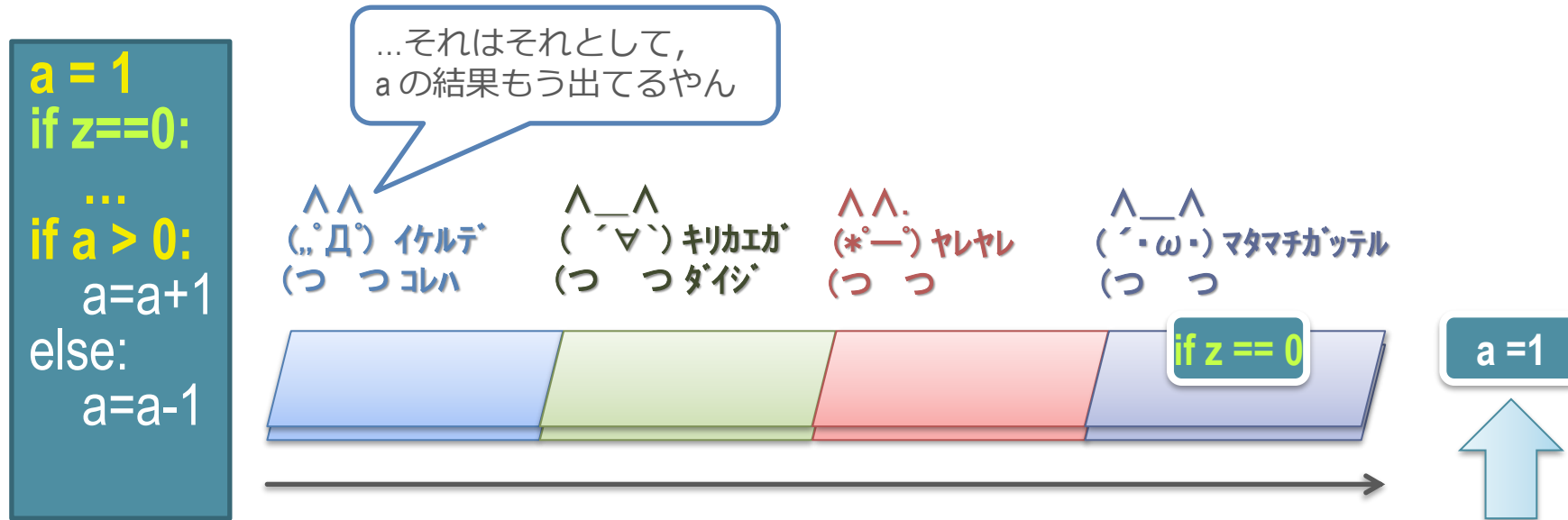


分岐命令の処理と制御ハザード



- 「if a > 0」の結果は最終段の(^_ ^)の人まで反映出来ない
 - 先頭は次に a=a+1 と a=a-1 のどちらを取り込めばいいのかわからない
- 当然に, a の値も予測時にはわかっていないから使えない

RBias のやりかた



- 連続した分岐の1個目（「if z==0」）でミスがあった場合を考える
 - 「if z==0」の後には一旦取り消される
 - このとき、その前に決定された値「a=1」が観測可能になる
 - 分岐予測ミスはバーストで起きやすいので結構こういう事がある
- 他にも関数の引数などで、長期間変わってない値も使える

ダイジェストの作り方と, RBias が効くコードの例

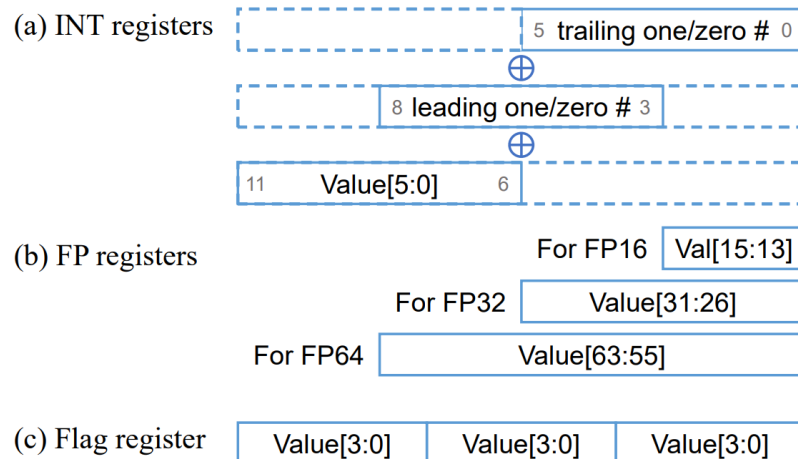


Fig. 4. Hash functions for generating digests from register values.

```
1 void qsort(void* a, size_t n, size_t es, /**/) {
2     if (n < 7) { ◀hard-to-predict branch
3         : /* insertion sort */
4         return;
5     }
6     : /* select a pivot based on the median of a few values */
7     pa = pb = (char*)a + es;
8     pc = pd = (char*)a + (n-1) * es; ◀calculate n-1
9     : /* 1st stage of 3-way partitioning, modifies pa through pd */
10    pn = (char*)a + n * es; ◀calculate n*es
11    : /* 2nd stage of 3-way partitioning */
12    if ((r = pb - pa) > es)
13        qsort(a, r / es, es, /**/ );
14    if ((r = pd - pc) > es)
15        qsort(pn - r, r / es, es, /**/ );
16 }
```

Fig. 5. Quicksort code that illustrates key behaviors of RBias.

- INT では MSB/LSB の連続 0/1 数や LSB 自体, FP では符号と指数から汎化性があるダイジェストを作成し, ハッシュに使用
- たとえばクイックソート末端でそろそろ再帰から戻るところなどを当てられる
 - 変数 n が小さくなってくると, 次のパターンが決まってくる

相関を抽出する機構

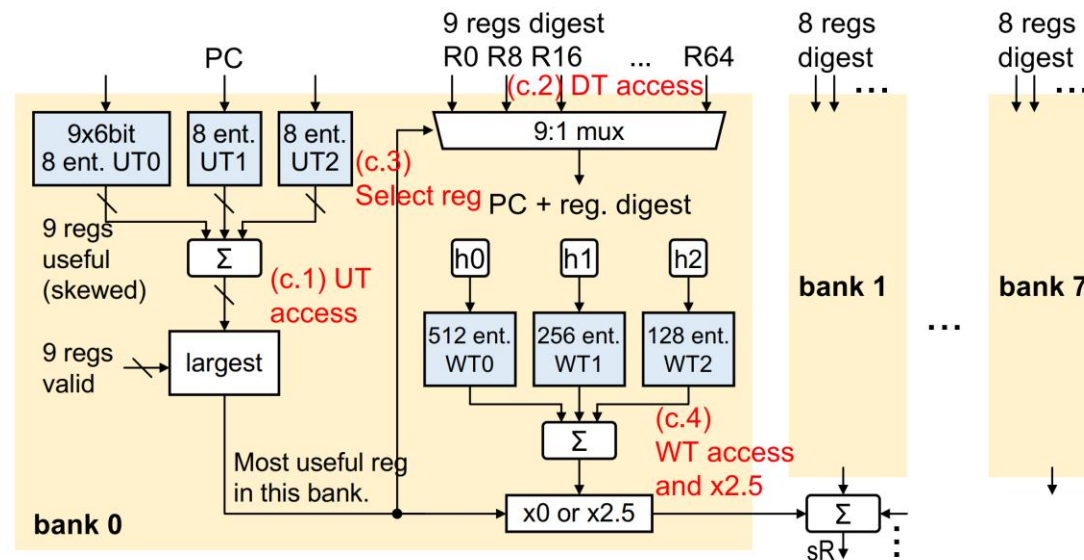


Fig. 3. Structure of RBias.

- 課題：
 - 基本的にはレジスタ値は分岐結果と無関係のノイズまみれで、相関を拾うのは大変
 - 常時全レジスタを見張るのも厳しい
- バンク化やランダムサンプリング，結果の増幅をうまく使ってコンパクトに実現
 - 「こう言うところに相関があるはずだ」を思いついたのは塩谷だが，実際に相関をとれる機構を作ったのは小泉くん（とてもえらい

予測ミスしたパスからの相関の抽出

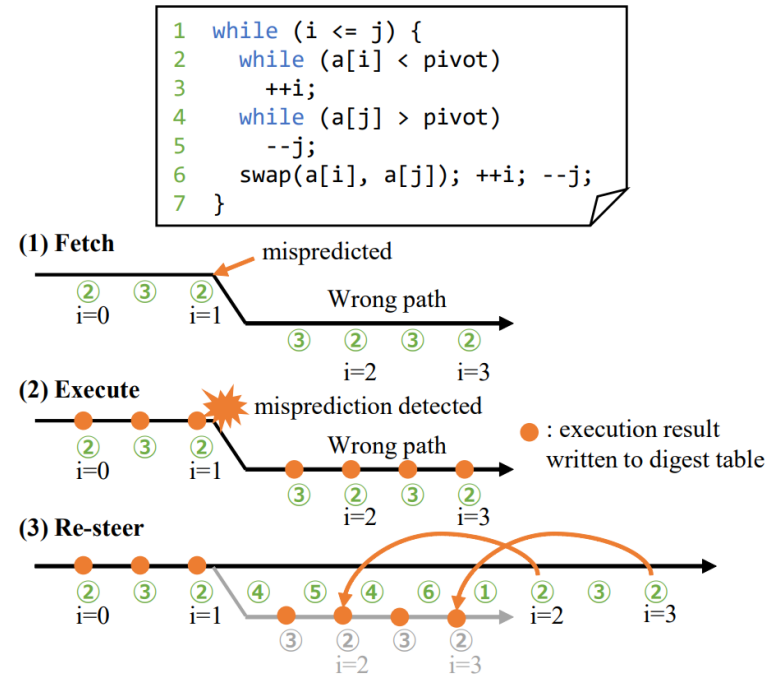


Fig. 7. Seq-RBias capturing a correlation with a value generated on a wrong path. The circled numbers correspond to source-code line numbers.

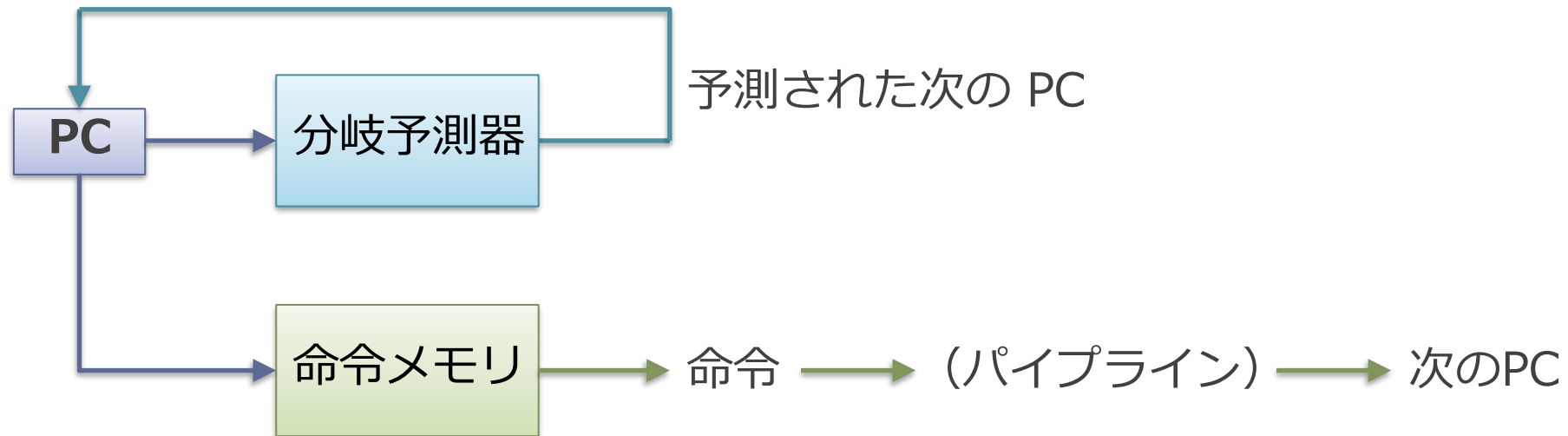
- Seq-RBias（発展型）では，予測ミスした実行パスの結果からも有用な情報を拾えるようになった
- リングバッファに格納して消すのを忘れてたら勝手に拾って改善してた

予測器とフェッチパイプライン

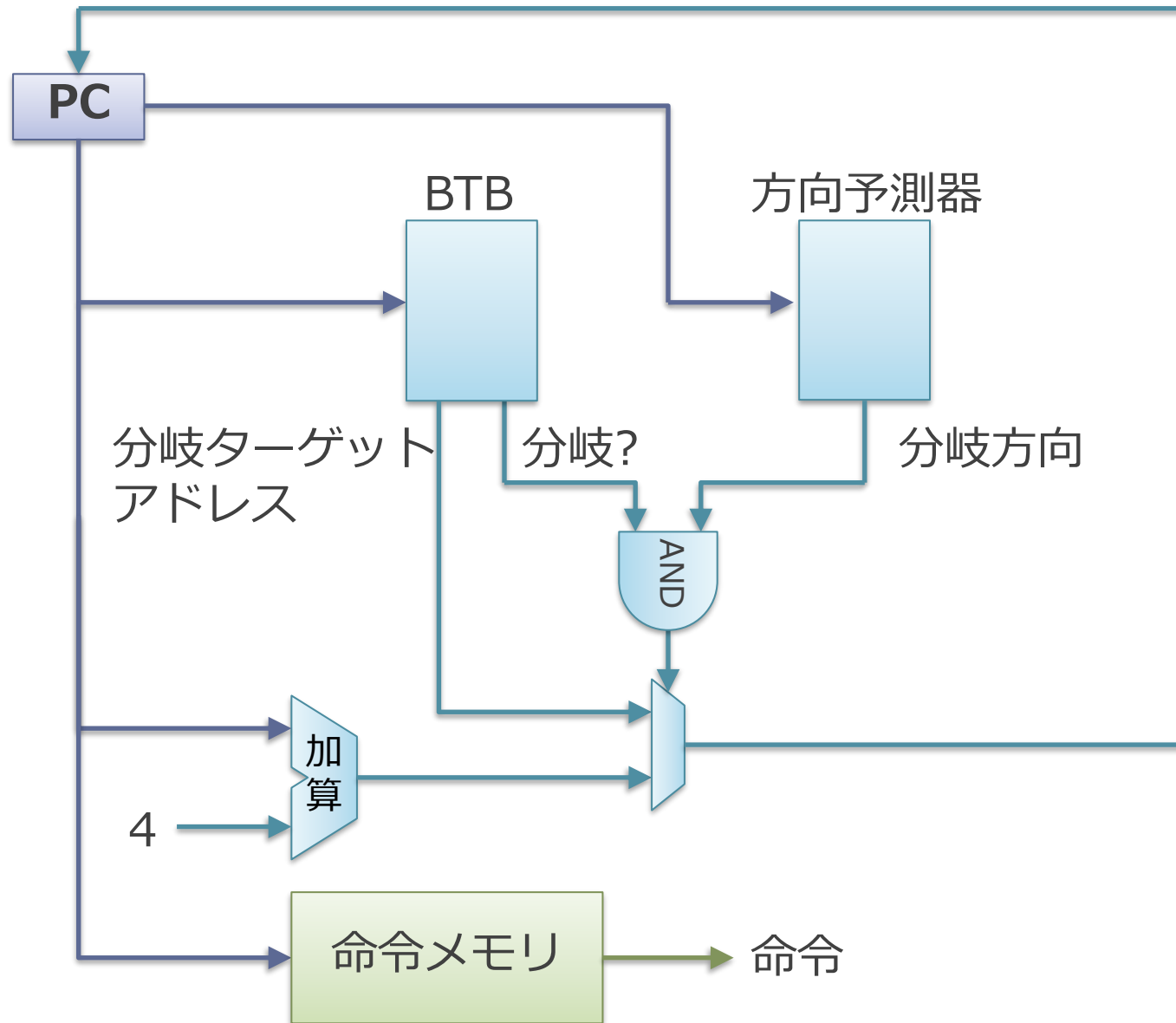
分岐予測器の全体構造

■ 分岐予測器全体の仕事：

- 現在の PC から次の PC を予測する
- 読み出された命令が実行されて次の PC が確定するのを待たずに次の PC を予測で決定する

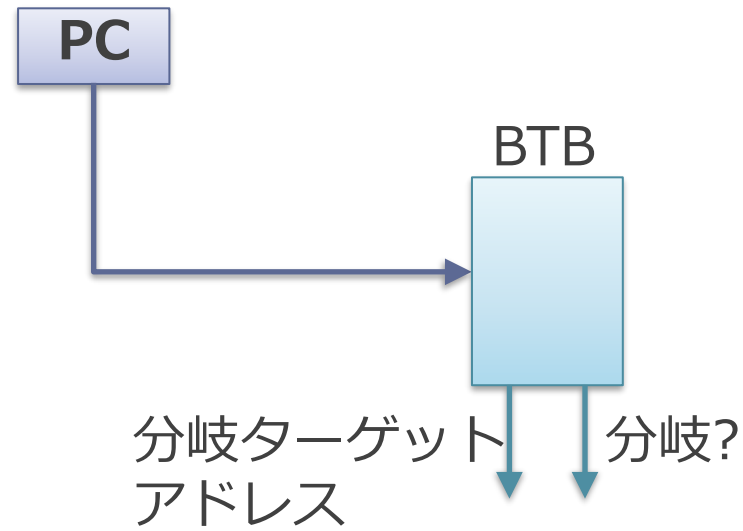


分岐予測器の全体構造



分岐予測器の動作（１）

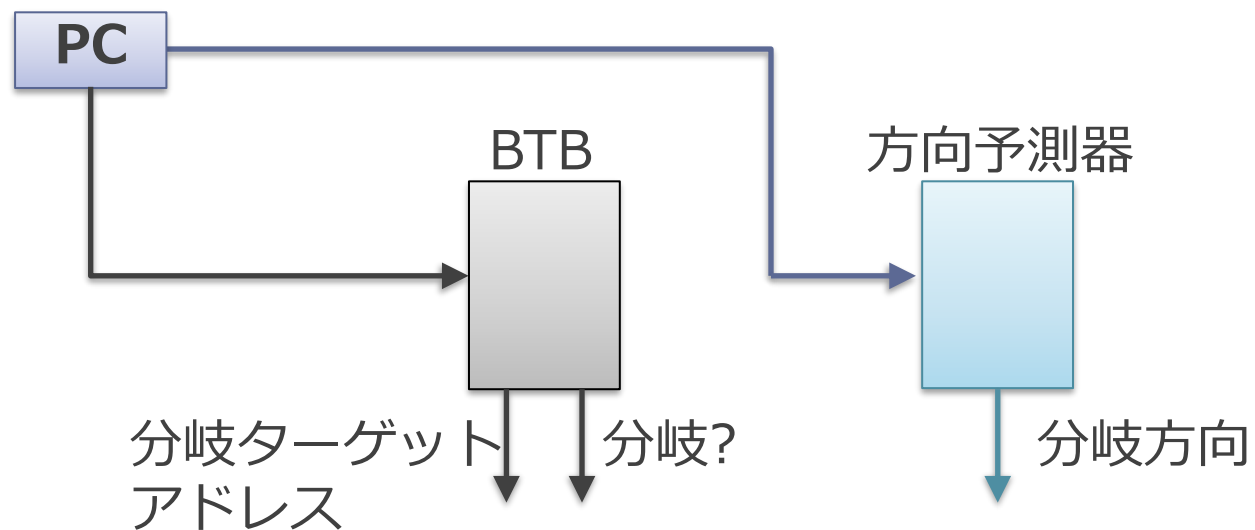
BTB による分岐ターゲットと分岐かどうかの予測



- BTB により以下を予測
 - 分岐の飛び先（ターゲット・アドレス）
 - 分岐かどうか

分岐予測器の動作（２）

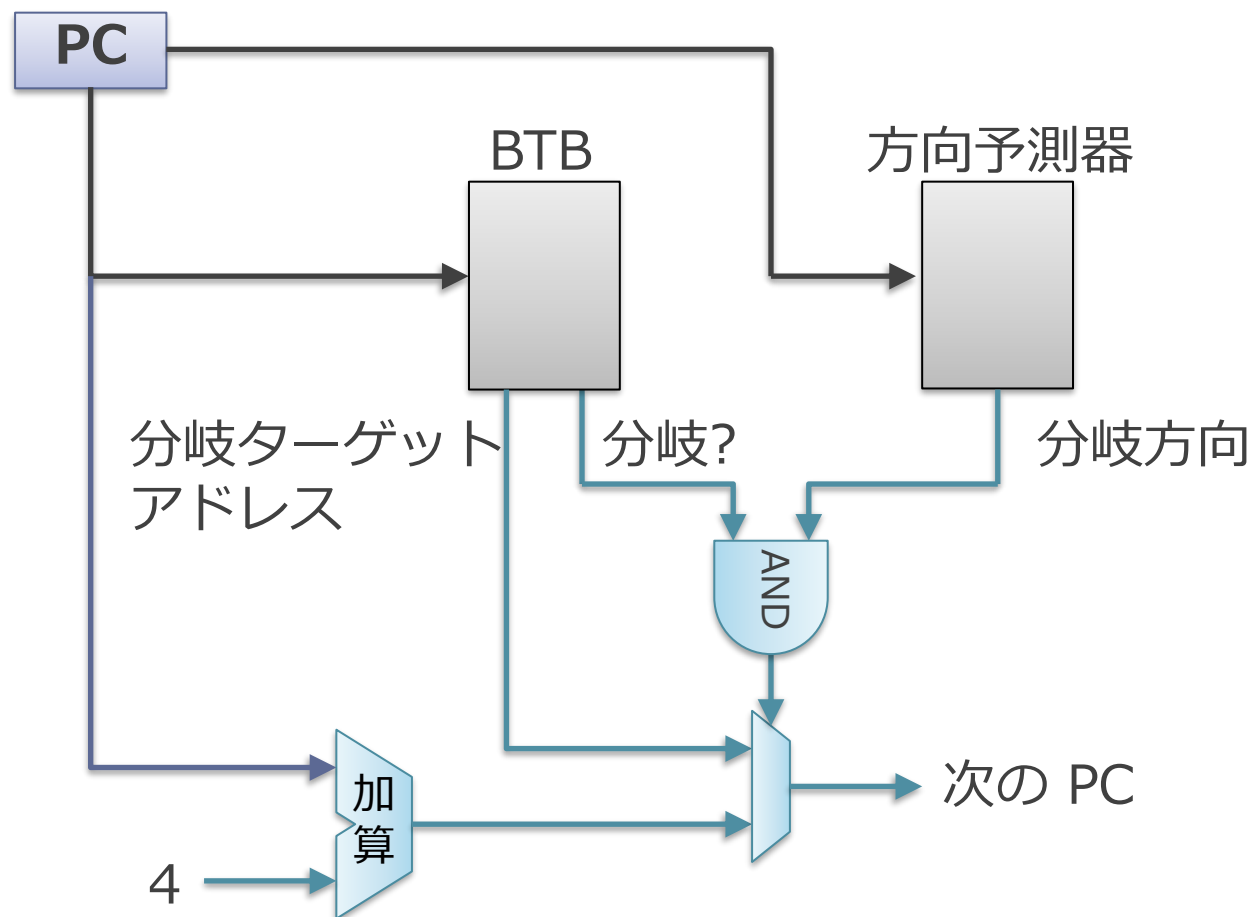
方向予測器による分岐方向の予測



- 方向予測器により，分岐の方向を予測

分岐予測器の動作 (3)

次の PC の予測



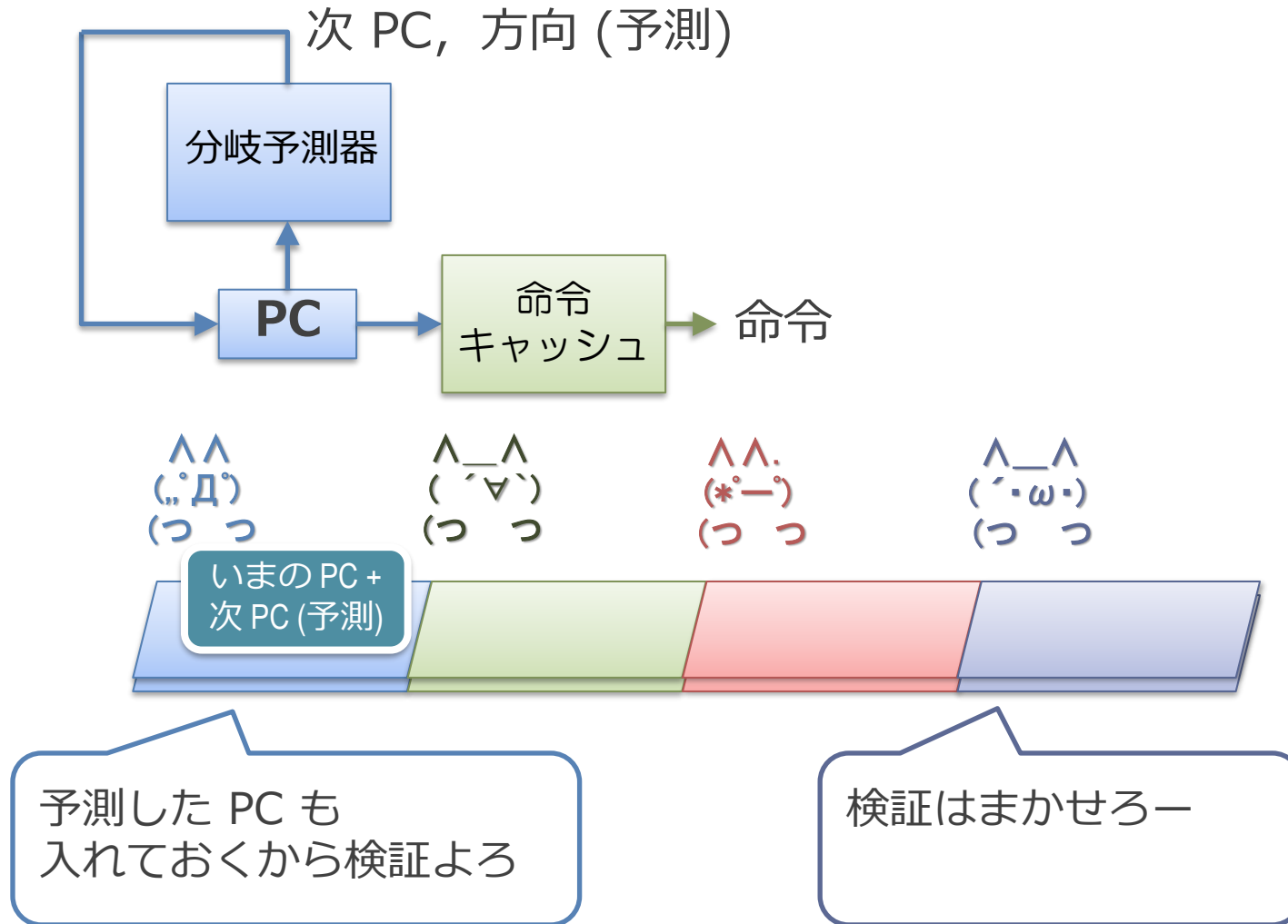
- 次の PC をマルチプレクサにより選択
 - 分岐命令かつ分岐が成立なら, ターゲット・アドレスを選択
 - そうでなければ $PC + 4$ を選択

分岐予測の基本の補足

1. 分岐予測器の全体構造
2. **パイプラインとしての動作**

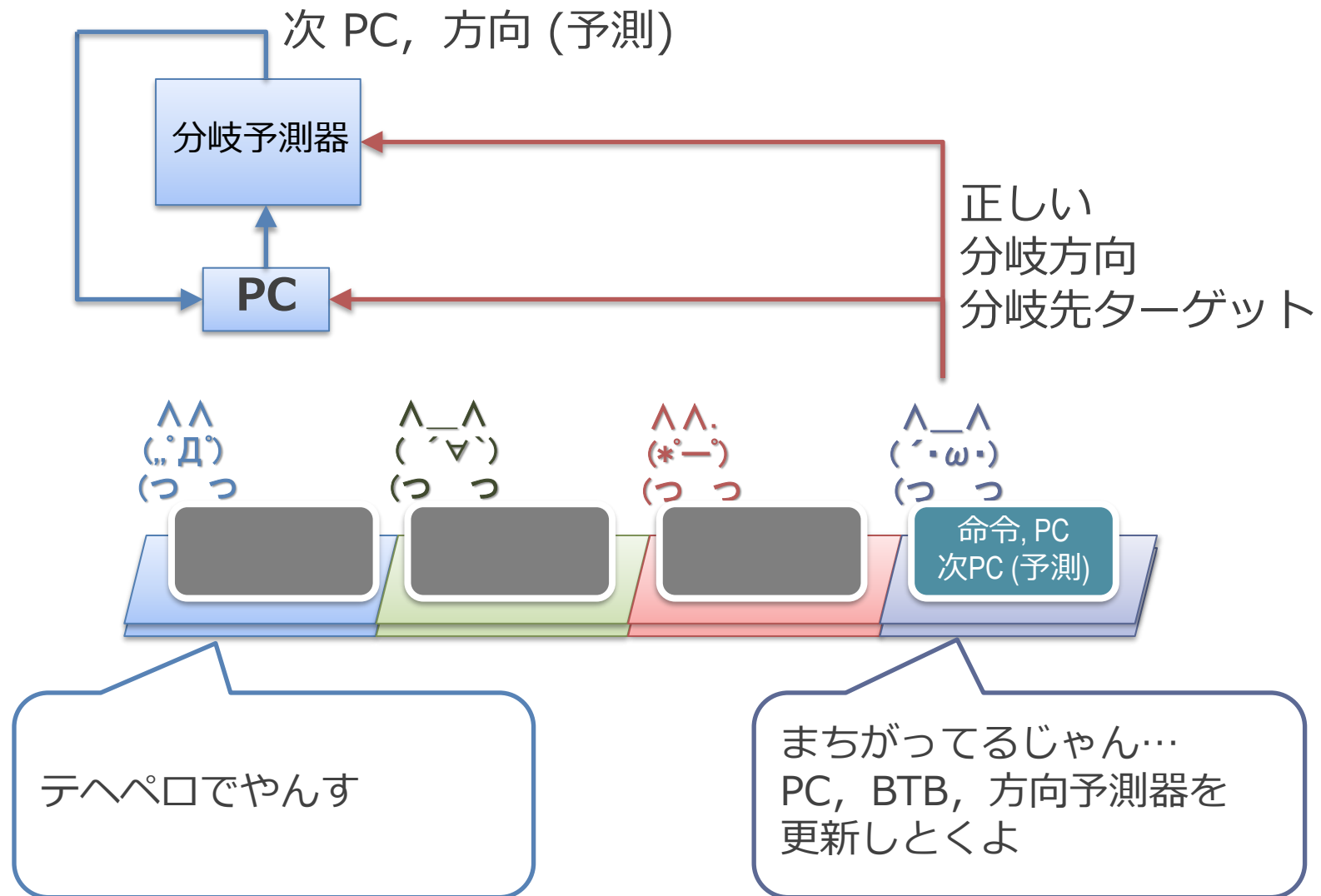
パイプラインとしての動作 (1)

予測結果の PC や方向をパイプラインに流す

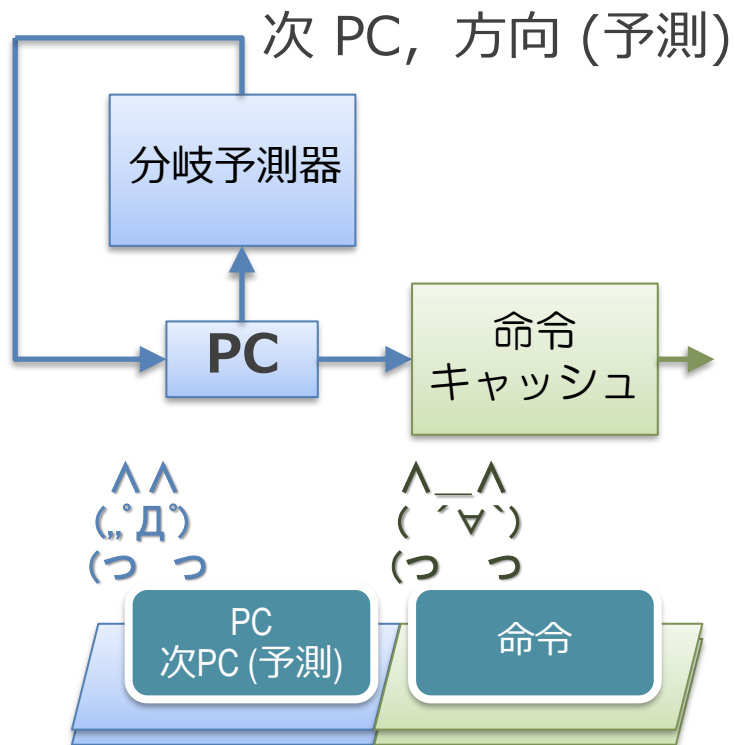


パイプラインとしての動作 (2)

予測ミス判明時に予測器や PC を学習



分岐予測器を含む命令フェッチパイプラインの課題



■ 課題 :

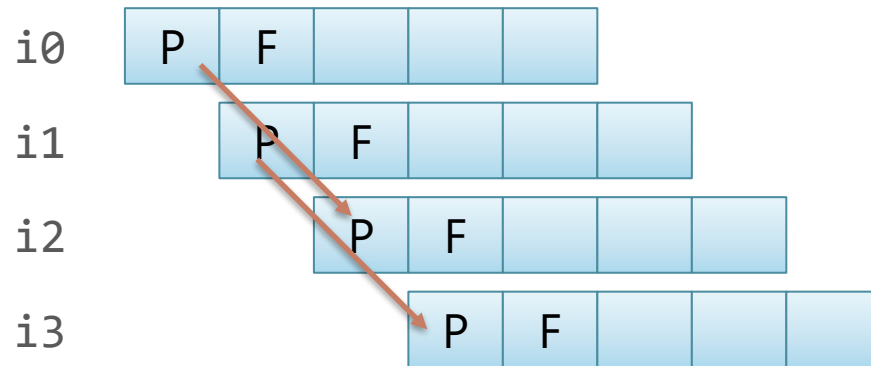
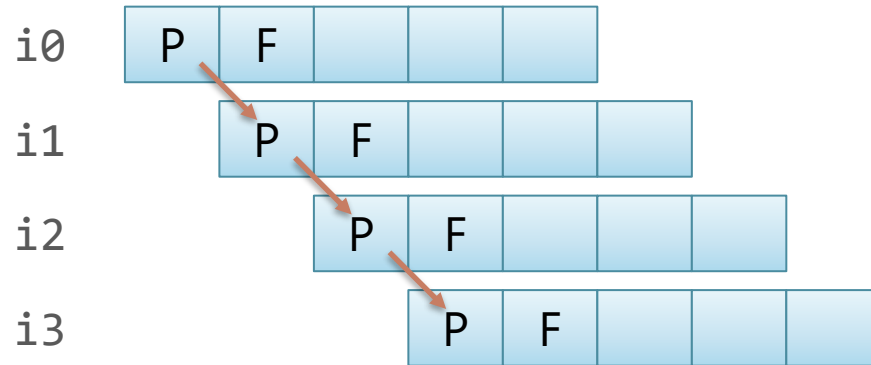
1. 分岐予測器は 1 サイクル以内に次の予測を返さないといけない
 - 分岐予測器は PC を入力に次の PC を予測する
 - ◇ 自分自身の出力に依存している
 - 2 サイクルかける = パイプラインにバブルが入る
2. 命令キャッシュがミスすると分岐予測器もとまる
 - 予測器はキャッシュの上流にあるため

■ 提案 :

1. Ahead Pipelining
2. Decoupled Frontend

Ahead Pipelining

- 1 サイクル前の PC を使って、今の次の PC を予測する



■ 通常の予測パイプライン

- 1 サイクル前の命令の PC から次の命令の PC を予測
- $i0 \rightarrow i1$, $i1 \rightarrow i2$

■ Ahead Pipelining

- 2 サイクル前の命令の PC から次の命令の PC を予測
- $i0 \rightarrow i2$, $i1 \rightarrow i3$
 - 交互にインタリーブされている

Ahead Pipelining の効果

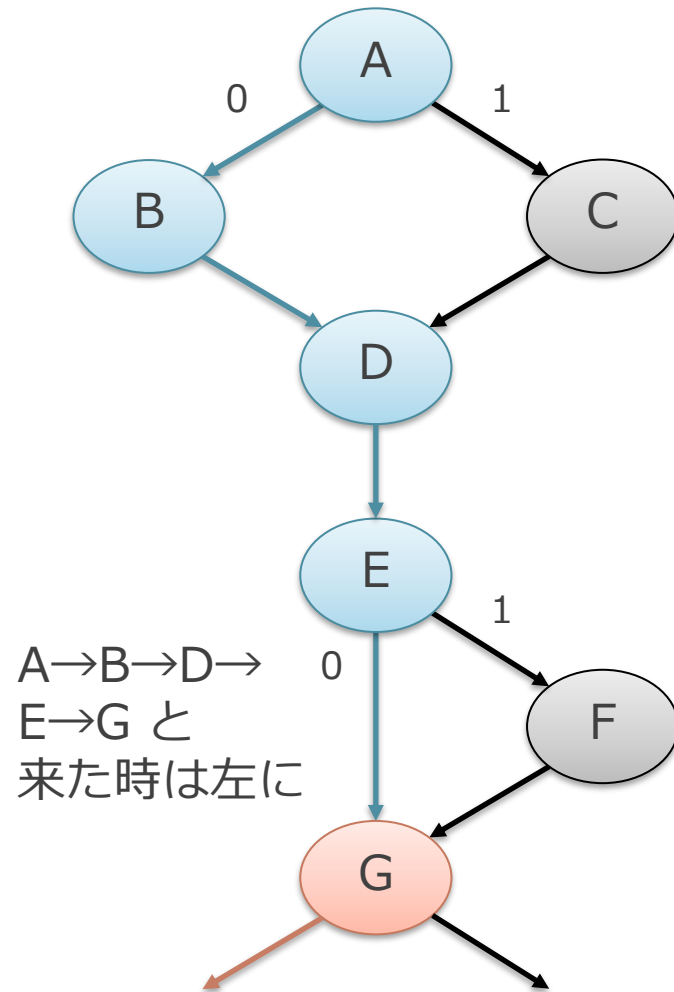
■ 利点：

- 予測を複数サイクルかけて実施可能になる
- 大きな予測器が使用可能になる
 - 方向予測器にも BTB にも使える

■ 問題：

- 予測精度が下がる

Ahead Pipelining の課題



■ 通常のパイプライン :

- E に来た時に F or G を予測

■ Ahead Pipelining した場合 :

- A に来た時に, E が F or G を予測
- 問題 : より遠い未来の予測になる
 - B にいったのか C にいったのかの情報が使えない
 - C が分岐だった場合, E ではない可能性すらある

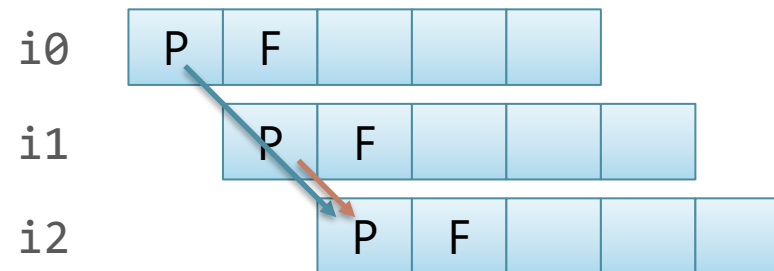
Ahead Pipelining の課題の緩和方法

■ 直近の情報もあわせて使う

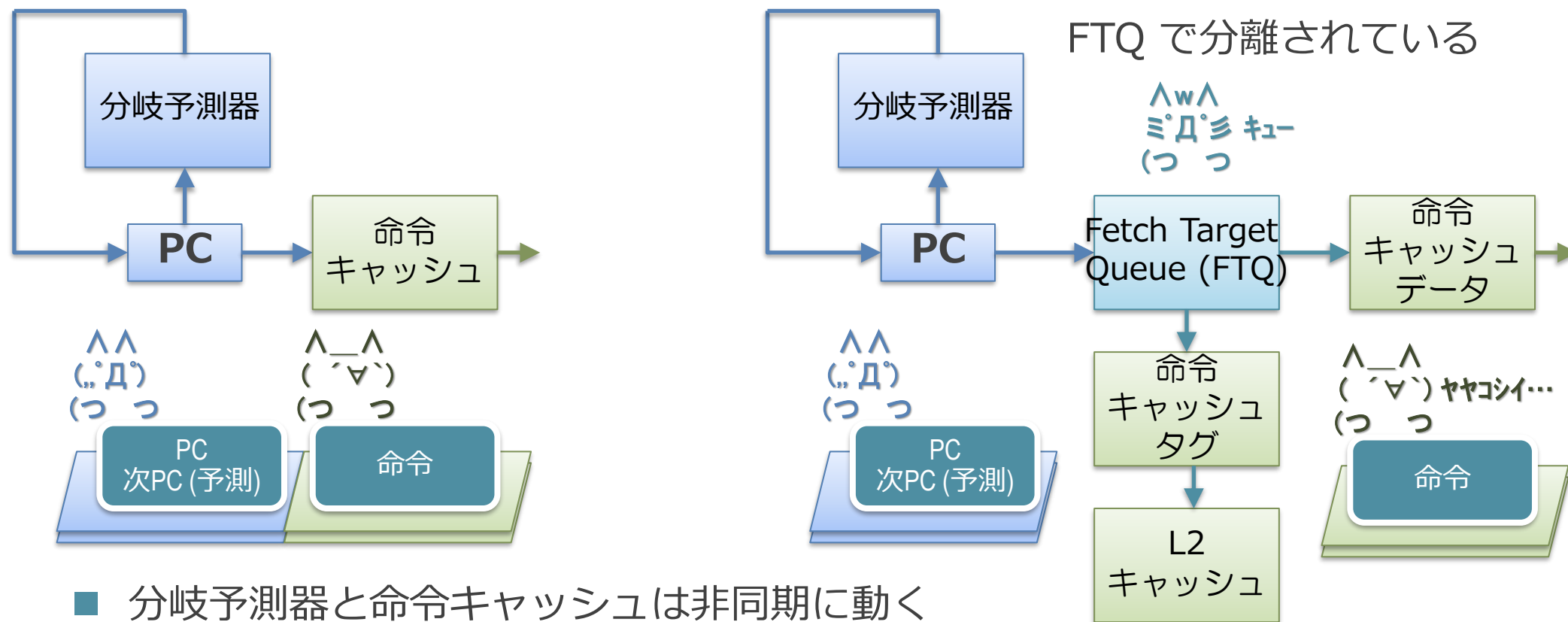
- たとえば BTB で i2 の予測を行う際に,
 - テーブルの読み出しに使うインデクスは PC (i0)
 - 読み出したタグの比較は PC (i1) で行う
- TAGE であれば, 履歴で同様のことをする

■ 意味合い :

- 将来到達しうる場所の情報を, 時間をかけて複数読み出しておく
- その中から, 直近の情報を使って短時間で選択

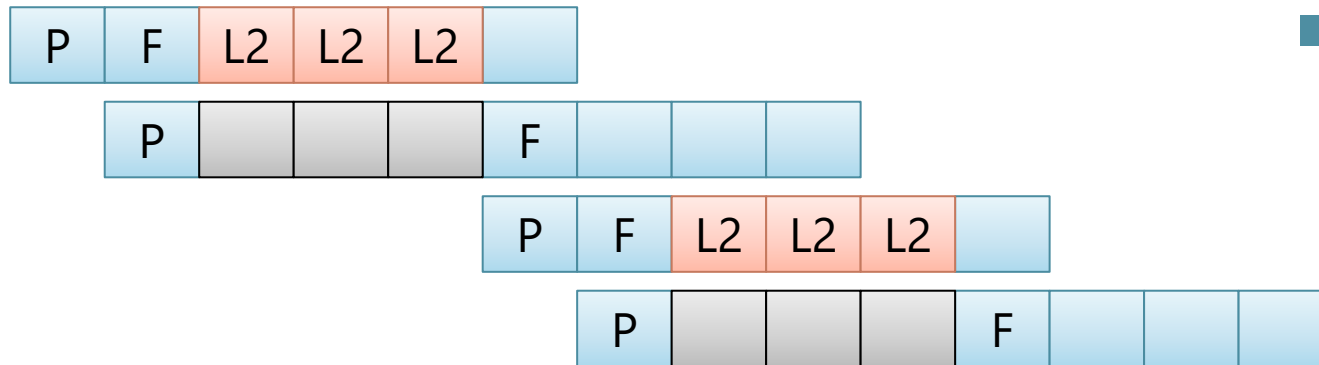


Decoupled frontend (右側)

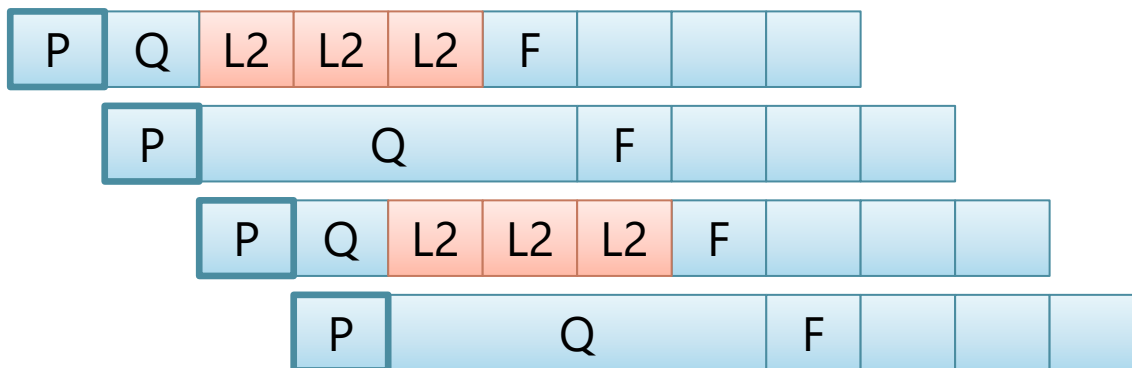


- 分岐予測器と命令キャッシュは非同期に動く
 - 分岐予測器はヒットミスに関係なく常にアドレスを生成し、キューイング
 - タグアクセスをしてヒットミス確認をし、ミスなら L2 にリクエスト
 - L2 から届くのを待たずに後続の処理を続行
 - 命令の読み出しは in-order にバッファないしはキャッシュから行う

Decoupled frontend の効果



- 通常のパイプライン：L2 が来るたびにバブルが入り全体がとまる



- Decoupled frontend
 - 予測 (P) は常に動き続ける
 - L2 のキックを即座に開始でき、L2 アクセスをオーバーラップできる

まとめ

分岐予測のまとめ

1. 予測精度と性能の簡単な見積もり
2. 間接分岐予測器
3. 高度な方向予測器
4. フェッチパイプライン

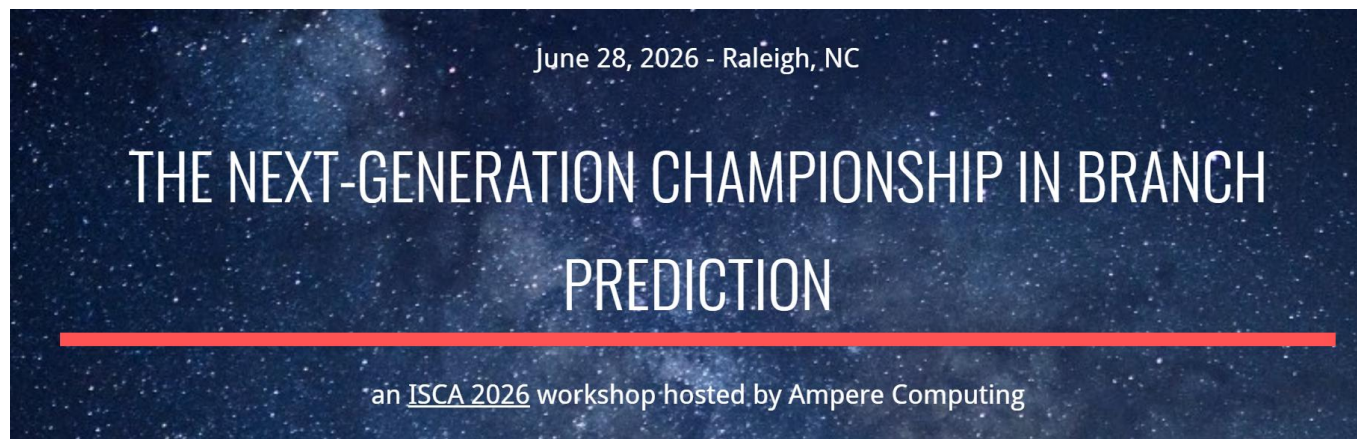
分岐予測のまとめ

- 「付録1:分岐予測の詳細」 (github 上にある) の内容 :
 - パーセプトロン予測器や TAGE 予測器の詳細
 - Piece-wise パーセプトロン, O-GEHL
 - BATAGE (Bayesian TAGE)
 - 予測ミスからの回復の実装方法
 - 複数命令を同時にフェッチする場合のやりかた
- (正直, 今見返すと情報が古かったり浅かったりするので更新予定)

TAGE の詳細と参考

- ここに書ききれない色々な話がまだかなりある...
 - 基本的なテーブルの学習順序
 - U-bit を使った置き換え
 - 最長履歴マッチと2番目のマッチの併用, そのための meta 予測
 - 超長履歴の高速な畳み込みの方法
 - 複数分岐命令予測時の履歴の作り方やテーブルの読み方
- TAGE: an engineering cookbook
 - <https://hal.science/hal-04804900>
 - Seznec 先生が, 実際に作るならという観点で色々な技をまとめている

興味がある人向け



- CBP-2025 : <https://ericrotenberg.wordpress.ncsu.edu/cbp2025/>
 - 去年久々に開催, こっちの方がとっつきがよい
- CBP-NG (2026) : <https://cbp-ng.bpchamp.com/>
 - 去年久々にやったら盛り上がったからか, 今年も開催予定
 - 回路レベルの事を考慮が入るなど, 伝統的なものから大幅にルールを変更
- 各種アルゴリズムを評価するシミュレータや入力データなどを配布

出欠と感想

- 本日の講義でよくわかったところ, わからなかったところ, 質問, 感想などを UTOL の出席欄に書いてください
 - LMS の出席を設定するので, そこにお願いします
 - パスワード : arc
- 意見や内容へのリクエストもあったら書いてください
- UTOL の出席の締め切りは来週の講義開始までに設定してあります
 - 仕様上「遅刻」表示になりますが, 特に減点等しません
 - 来週の講義開始までは感想や質問などを受け付けます

- 以下のスプレッドシートの自分の席のところに学籍番号を書いてください
 - https://docs.google.com/spreadsheets/d/1Q0hVjHBbNNdJb4Bs6RgyzoCilHbUWb4uWzlagx1BrYs/edit?usp=drive_link
 - 学生側から見た場合の配置で（画面上が教団側），席の対応する行と列のところに書いてください

- 古い資料にあったものを移動

ローカル・グローバル・ハイブリッド予測器

- ローカル予測器とグローバル予測器のそれぞれが得意な分岐がある
 - 基本的にはグローバルの方が強い
 - グローバル予測機はローカルなパターンもうまく予測できる
 - ローカルが得意な分岐の例：間隔が長い場合
 - ある関数内の if 文は成立と不成立を交互に繰り返す
 - その関数はかなり時間をあけて呼ばれる
 - グローバル予測器では相当長い履歴が必要
- アプローチ
 - ローカル予測器とグローバル予測器を両方積んで、使い分ける

ローカル・グローバル・ハイブリッド予測器

■ 要素

1. ローカル予測器
2. グローバル予測器
3. セレクタ
 - PC をインデクスとしてアクセスされるカウンタのテーブル

■ 学習の動作

- 1. ローカル予測器と 2. グローバル予測器で並列に予測
- セレクタの更新
 - 1. が当たってたらセレクタの対応エントリをデクリメント
 - 2. が当たってたらセレクタの対応エントリをインクリメント

■ 予測時の動作

- セレクタの対応エントリと閾値を比較してどちらを使うか決定

ローカル・グローバル・ハイブリッド予測器

- 問題点：容量効率が悪い
 - ローカルとグローバルが二重に存在
 - セレクタが追加される
- それほど予測精度は改善しない
 - しかし機構が割と単純なので結構いろんな CPU に乗っていた

質問や感想

- 3ビット以上に増やすと逆に「過去の学習に引っ張られすぎて、本当に全体の傾向が変わったときの適応が遅れる（精度が落ちる）」という指摘が非常に興味深かったです。計算機的设计でありながら、どこか「過去の成功体験に執着しすぎると変化に対応できない」という人間の心理や柔軟性のバランスに通ずるものがあったかもしれません(?)。

- 分岐予測は実装をしたことがないのですが、プログラムの分岐にはfor文も多く、単純なカウンタでも十分に予測が可能であるということがわかりました。ハードウェア側では性能の関係であまり大掛かりなことは出来ないと思うのですが、ソフトウェア側に記述の工夫で、分岐予測が成功しやすくなるようなものが存在するのかが気になりました。
- 最近分岐の性能予測が上がっているために問題にならないと思うが、CPUの性能コンテスト見たいなレベルまで行くと、分岐予測の際に基本的に成立するようなコードを書く方が分岐予測の精度も結果的に上がるのではないかと思った。
 - とにかく PC が直進する（分岐で飛ばない）ようにするのがよい

- 自分はHPCの研究室にいてハードウェアのパイプラインとかまで深ぼったアーキテクチャまではそこまで意識しないが、これからは分岐予測に易しい分岐命令を書いてあげたいと思った。

- 理論的なアルゴリズムの分野に比べて、ハード関連のアルゴリズムはより複雑というか経験則を利用したり現実上の課題を回避しなければならない制約があったりして、難しそうではありつつもその分アイデアや工夫の幅も広く感じる。
- 理想的な想定のもとだと棄却されるアルゴリズムでも制約のもとではむしろ最も実用的になるということだろうが、制約があるからこそ幅が広がるというのは面白い。

- D-FF自体の遅延の問題でパイプラインの高速化に限界があるというお話でしたが、パイプラインを用いるラッチをD-FF以外のラッチを用いたりした場合に多少改善したりすることができるのでしょうか？それともD-FFが1番最良のラッチになるのでしょうか？

- JAGUARのパイプラインの図でFetchが最大5段、Decが3段あって、フロントエンドは複雑なんだなぁと思った

- 現代の高性能プロセッサのパイプライン段数が15~20段という話がありましたが、、今後もこのあたりの段数で維持される見込みなんでしょうか？半導体の微細化やアーキテクチャの複雑化などで、最適な段数も結構変わりそうだと思います

- compilerにあるlikely, unlikely flagはコンパイラが分岐予測のヒントを与えるものと聞き, 実際にjemallocなどのソースコードで見かけたのですが, これはスライド中にある通り今は書かないようなコードということでしょうか?

- n ビットカウンタで、 n を3以上にすると性能が上がらない理由がよくわかりません。説明の中では、成立(or不成立)と強く(or弱く)予測するのかで4パターンの状態遷移を考えていると思いますが、2次元平面的に予測、つまりx軸には事象Aが成立するか否か、y軸に事象Bが成立するか否かみたいな感じでの2(以上の)ビットカウンタを設定することは難しいのでしょうか？

- パイプラインのステージ数が多くても20というのは、思ったより少なく感じました。一つ一つの要素は高度なのだと思うのですが、開発段階において各開発者が自分の担当領域に集中できるような抽象化をしているのでしょうか。

- 「どちらのホームに降りるか」という例は少し違う、という話をされていたかと思うので、他の正しい例を自分でも考えてみたのですが、日常生活では外すリスクを伴うものの方が圧倒的に多く、難しかったです。

- n ビット・カウンタ予測器でカウンタのビット数が2ビットの場合が性能のピークっぽいのは何か理論的な話があるのですか？それとも経験則的な話なのでしょうか？

- 命令に分岐のヒントを埋め込むことができるのであれば、コンパイラなどのソフトウェアに分岐予測を任せずに、プロセッサ側に行わせる意義は何だろうという疑問に思った。

- 分岐予測器は、他のパイプラインの処理と比較して結構リッチな処理を行っている印象があり、cycle数も大きそうな直感を抱きました。専用の回路になっているなどして、高速に処理しているのでしょうか？

- クロック信号が複数のnotで生成されているという話を聞いて、実際のCPU/FPGA上でもnotが置かれているのかが気になりました. clk信号生成用の専用パーツが乗っているのか, それかそんなコストかからないしLUTやらに合成されるのか.....

- 前半では、周波数向上を妨げる要因に対する理解が深まったように思う。パイプラインレジスタ内部のゲート遅延で決まる上限についてまだ10倍程度の余裕があることを鑑みると、2回前の講義で紹介された考え方である「電圧の下限に到達→電力密度の増加を避けるために周波数を保持」の方がより差し迫った問題であることのイメージも湧いた。一方で、近年のアーキテクチャ分野における消費電力削減の研究・議論では、どの程度、動作周波数の向上が研究動機として念頭に置かれているのかが気になった。

- よく MEM stage がメモリアクセスするステージとして扱われますが、fetch もメモリアクセスなので、両者のパイプラインの分割数とかは大体類似しているものとして捉えていいのか、何かしら非対称性があるのかどうなんでしょうか。

- 予測分岐について初めて学習したが、とても難しいことをやっているのだと驚いた。1サイクル以内にBTBやPHTをひいてその値を取得するのが可能なことについて、そんなに高速に辞書を引けることを不思議に感じた。カイジネタが差し込まれていて面白かった。

- ハードウェアのマルチスレッドと、ソフトウェアのマルチスレッドが違うというのは、私にとっては結構驚きでした。今までこの2つを混同して使っていた場面もあったので。

- (GPU回で話がありそうですが、)GPUにも簡単な条件分岐の仕組みは載ってたりするのか、あるいはその試み等がありますか？